

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CMC**



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG WEB (PROG2008)

Tên đề tài:
**WEB ĐỌC TRUYỆN CHỮ
ONLINE-CMC TRUYỆN**

Giảng viên: Nguyễn Đức Giang

Mã lớp học phần: N03

Nhóm sinh viên:

1. Trần Thị Kim Uyên BAI250072
2. Nguyễn Thị Thùy BAI252513
3. Nguyễn Hải Dương BAI250020
4. Nguyễn Tuấn Thành BAI252417

Hà Nội, tháng 7 – 2026

PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	Mã sinh viên	Nội dung phụ trách
1	Trần Thị Kim Uyên	BAI250072	Nhóm trưởng, Database Architect; quản lý backlog, thiết kế PostgreSQL schema, migration và ERD; phụ trách tìm kiếm, lọc và tối ưu truy vấn; phát triển AI, upload Supabase, collaborator và giao diện quản trị Admin.
2	Nguyễn Thị Thùy	BAI252513	Phát triển giao diện người dùng gồm trang chủ, tìm kiếm, chi tiết truyện, đọc truyện và lịch sử đọc; phụ trách sắp xếp, phân trang và kiểm thử nghiệm thu; tổng hợp báo cáo.
3	Nguyễn Hải Dương	BAI250020	Phát triển CRUD truyện, chương, validation và kiểm tra quyền; tích hợp API Frontend–Backend; xây dựng giao diện bình luận, follow, rating, thông báo và khu vực Moderator; kiểm thử component.
4	Nguyễn Tuấn Thành	BAI252417	Backend Lead; phát triển API danh sách, chi tiết, authentication, JWT, RBAC, OAuth/OTP, moderation, notification, audit log và Redis/BullMQ; phụ trách bảo mật, tối ưu API, DevOps, backend test và tài liệu kỹ thuật.

MỤC LỤC

DANH MỤC BẢNG	iii
DANH MỤC HÌNH	iv
Danh mục từ viết tắt	v
1 GIỚI THIỆU BÀI TOÁN	1
1.1 Bối cảnh và động cơ thực hiện	1
1.2 Phát biểu bài toán	1
1.2.1 Vấn đề của người đọc	1
1.2.2 Vấn đề của người đăng nội dung và đội ngũ vận hành	1
1.3 Mục tiêu của hệ thống	2
1.4 Đối tượng sử dụng và bên liên quan	2
1.5 Phạm vi chức năng	3
1.5.1 Chức năng trong phạm vi	3
1.5.2 Ngoài phạm vi hoặc chưa đủ minh chứng	4
1.6 Yêu cầu phi chức năng	5
1.7 Phương pháp thực hiện	6
1.8 Kết luận chương	6
2 PHÂN TÍCH LUỒNG HOẠT ĐỘNG	7
2.1 Tổng quan tác nhân và nguyên tắc phân quyền	7
2.2 Luồng của khách và người dùng	8
2.2.1 Khám phá và đọc truyện khi chưa đăng nhập	8
2.2.2 Đăng ký, đăng nhập và duy trì phiên	9
2.2.3 Theo dõi, lịch sử và tương tác	10
2.3 Luồng của Uploader	11
2.3.1 Tạo truyện và nhập nội dung	11
2.3.2 Quản lý chương và cộng tác viên	11
2.4 Luồng của Moderator	12
2.5 Luồng của Admin	13
2.6 Luồng báo cáo vi phạm	14
2.7 Truy vết luồng đến endpoint và source	15
2.8 Quy tắc xử lý lỗi và an toàn luồng	16
2.9 Kết luận chương	16

3	TRIỂN KHAI HỆ THỐNG	17
3.1	Kiến trúc phần mềm tổng thể	17
3.1.1	Lựa chọn mô hình kiến trúc	17
3.1.2	Công nghệ và trách nhiệm	18
3.2	Triển khai frontend	19
3.2.1	Cấu trúc thư mục	19
3.2.2	Điều hướng và bảo vệ route	20
3.2.3	Lớp gọi API và quản lý phiên	21
3.2.4	Trạng thái giao diện và trải nghiệm	22
3.3	Triển khai backend	22
3.3.1	Cấu trúc phân lớp	22
3.3.2	Pipeline request	23
3.3.3	Xác thực, phân quyền và audit	25
3.3.4	Tác vụ nền và dịch vụ AI	26
3.4	Triển khai cơ sở dữ liệu	26
3.4.1	Mô hình dữ liệu	26
3.4.2	Connection pool, index và transaction	29
3.4.3	Nợ kỹ thuật dữ liệu	30
3.5	Cấu hình và triển khai	30
3.5.1	Biến môi trường	30
3.5.2	Topology triển khai	30
3.6	Kiểm thử và kết quả xác minh	32
3.6.1	Kết quả thực chạy	32
3.6.2	Kịch bản kiểm thử thủ công cần bổ sung	32
3.7	Giao diện người dùng	33
3.7.1	Giao diện khách hàng	34
3.7.2	Giao diện Admin và Moderator	38
3.8	Đánh giá, hạn chế và hướng hoàn thiện	41
3.8.1	Điểm đạt được	41
3.8.2	Hạn chế cần công khai	42
3.8.3	Thứ tự ưu tiên hoàn thiện	42
3.9	Kết luận chương	42
	Kết luận	43
	Tài liệu tham khảo	44

DANH MỤC BẢNG

1.1	Mục tiêu của hệ thống	2
1.2	Đối tượng sử dụng và nhu cầu chính	3
1.3	Yêu cầu phi chức năng và giải pháp đáp ứng	5
2.1	Ma trận quyền theo vai trò được thể hiện trong source	7
2.2	Luồng quản trị chính và dấu vết cần lưu	13
2.3	Truy vết luồng nghiệp vụ đến API và module hiện thực	15
3.1	Stack công nghệ được xác nhận từ package và cấu hình dự án	18
3.2	Trách nhiệm các thư mục frontend chính	19
3.3	Phân lớp backend và nguyên tắc phụ thuộc	23
3.4	Nhóm biến môi trường và nguyên tắc bảo mật	31
3.5	Liên kết bàn giao và minh chứng triển khai	31
3.6	Kết quả kiểm thử và build đã thực chạy	32
3.7	Kịch bản kiểm thử thủ công trước khi nộp	33

DANH MỤC HÌNH

1.1	Sơ đồ ngữ cảnh và các tác nhân chính	2
2.1	Luồng khám phá và đọc truyện của Guest	8
2.2	Trình tự xác thực và sử dụng JWT	9
2.3	Vòng đời kiểm duyệt của truyện	11
2.4	Luồng Moderator xử lý truyện chờ duyệt	12
3.1	Kiến trúc logic của hệ thống theo source hiện tại	17
3.2	Cấu trúc thư mục frontend	19
3.3	Đoạn code chính về route và route guard ở frontend	20
3.4	Đoạn code chính về Axios interceptor ở frontend	21
3.5	Cấu trúc thư mục backend	22
3.6	Pipeline xử lý request ở backend	23
3.7	Đoạn code chính cấu hình middleware và mount router	24
3.8	Đoạn code chính về xác thực JWT và RBAC	25
3.9	Lược đồ ERD của cơ sở dữ liệu CMC Truyện	27
3.10	Cấu trúc script, migration và schema cơ sở dữ liệu	28
3.11	Đoạn SQL chính về bảng truyện, chương và ràng buộc	29
3.12	Giao diện trang chủ dành cho khách hàng	34
3.13	Giao diện tìm kiếm và khám phá truyện	35
3.14	Giao diện chi tiết truyện	36
3.15	Giao diện đọc chương và tùy chọn đọc	37
3.16	Giao diện cài đặt cá nhân	38
3.17	Giao diện dashboard quản trị	39
3.18	Giao diện quản lý người dùng	39
3.19	Giao diện quản lý bình luận	40
3.20	Giao diện quản lý báo cáo vi phạm	40
3.21	Giao diện nhật ký thao tác quản trị	41

DANH MỤC TỪ VIẾT TẮT

AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CSDL	Cơ sở dữ liệu
CSS	Cascading Style Sheets
DOM	Document Object Model
FE	Frontend – Phần giao diện phía người dùng
BE	Backend – Phần xử lý phía máy chủ
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
RBAC	Role-Based Access Control – Kiểm soát truy cập theo vai trò
REST	Representational State Transfer
SPA	Single-Page Application – Ứng dụng web một trang
SQL	Structured Query Language
UI/UX	User Interface/User Experience – Giao diện/Trải nghiệm người dùng
URL	Uniform Resource Locator

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1. Bối cảnh và động cơ thực hiện

Nhu cầu đọc truyện trực tuyến ngày càng gắn với khả năng tìm kiếm nhanh, tiếp tục đúng vị trí đã đọc và sử dụng ổn định trên nhiều thiết bị. Khi số lượng truyện, chương, bình luận và báo cáo tăng, cách quản lý thủ công dễ phát sinh chậm trễ, sai sót và khó truy vết.

Từ thực tế đó, nhóm xây dựng ứng dụng web **CMC Truyện**. Hệ thống hướng đến hai mục tiêu chính: tạo trải nghiệm đọc thuận tiện cho người dùng và cung cấp quy trình đăng tải, kiểm duyệt, quản trị nội dung rõ ràng cho đội ngũ vận hành.

1.2. Phát biểu bài toán

1.2.1. Vấn đề của người đọc

Các vấn đề chính của người đọc gồm:

- khó tìm kiếm và chọn truyện khi dữ liệu tăng;
- không đồng bộ lịch sử, vị trí đọc và truyện đang theo dõi;
- thiếu tùy chỉnh hiển thị và thông báo chương mới;
- bình luận rác hoặc nội dung vi phạm làm giảm chất lượng cộng đồng.

1.2.2. Vấn đề của người đăng nội dung và đội ngũ vận hành

Người đăng nội dung cần quản lý truyện, chương và theo dõi trạng thái kiểm duyệt. Người kiểm duyệt cần xử lý hàng chờ, báo cáo vi phạm và phản hồi cho người đăng. Quản trị viên cần quản lý tài khoản, vai trò, nội dung và nhật ký thao tác. Vì vậy, hệ thống không chỉ thực hiện các chức năng thêm, sửa, xóa mà còn phải bảo đảm xác thực, phân quyền, quy trình trạng thái và khả năng truy vết.

1.3. Mục tiêu của hệ thống

Bảng 1.1: Mục tiêu của hệ thống

Mã	Mục tiêu	Kết quả mong đợi
M1	Cung cấp trải nghiệm khám phá và đọc truyện thuận tiện	Có trang tìm kiếm, chi tiết truyện, đọc chương, bảng xếp hạng và URL định danh theo slug.
M2	Cá nhân hóa trải nghiệm người dùng	Có lịch sử đọc, chương đã đọc, theo dõi truyện, thông báo và tùy chọn đọc.
M3	Bảo đảm quy trình xuất bản có kiểm duyệt	Truyện mới của Uploader ở trạng thái chờ; Moderator/Admin có thể duyệt, yêu cầu sửa hoặc từ chối.
M4	Bảo vệ thao tác theo đúng thẩm quyền	JWT xác thực phiên; middleware RBAC bảo vệ route; frontend có route guard tương ứng.
M5	Hỗ trợ quản trị và truy vết	Có quản lý người dùng, truyện, báo cáo, từ khóa nhạy cảm và audit log.
M6	Có khả năng kiểm chứng chất lượng kỹ thuật	Có kiểm thử backend, frontend và quy trình build production; kết quả thực chạy được trình bày ở Chương 3.

Báo cáo mô tả hệ thống tại thời điểm hoàn thiện sản phẩm. Kết quả được ghi nhận dựa trên mã nguồn, kiểm thử, bản build và minh chứng vận hành tương ứng.

1.4. Đối tượng sử dụng và bên liên quan



Hình 1.1: Sơ đồ ngữ cảnh và các tác nhân chính

Hình 1.1 khái quát năm nhóm tác nhân tương tác với hệ thống. Trong đó, Guest là người chưa đăng nhập, không phải một vai trò được lưu trong bảng users.

Bảng 1.2: Đối tượng sử dụng và nhu cầu chính

Đối tượng	Nhu cầu	Giá trị hệ thống cung cấp
Guest	Khám phá nội dung trước khi đăng ký	Xem trang chủ, tìm kiếm, bảng xếp hạng, thông tin truyện và nội dung công khai.
User	Đọc lâu dài và tương tác	Lưu lịch sử, theo dõi, đánh giá, bình luận, nhận thông báo và tùy chỉnh trải nghiệm đọc.
Uploader	Đưa nội dung lên hệ thống	Tạo và chỉnh sửa truyện, nhập nội dung, quản lý chương và cộng tác viên theo quy trình duyệt.
Moderator	Duy trì chất lượng nội dung	Duyệt truyện chờ và xử lý báo cáo, bình luận hoặc hồ sơ theo phạm vi được giao.
Admin	Quản trị toàn hệ thống	Quản lý người dùng, vai trò, truyện, báo cáo, từ khóa, nhật ký và số liệu tổng quan.
Nhóm phát triển	Bảo trì và mở rộng sản phẩm	Duy trì kiến trúc, API, cơ sở dữ liệu, kiểm thử, tài liệu và cấu hình triển khai.

1.5. Phạm vi chức năng

1.5.1. Chức năng trong phạm vi

Phạm vi chức năng được xác định từ yêu cầu dự án, route frontend và router backend [1], gồm:

- **Xác thực và tài khoản:** đăng ký, đăng nhập, đăng xuất, Google OAuth, quên/đặt lại mật khẩu bằng OTP và cập nhật hồ sơ;
- **Khám phá truyện:** danh sách, tìm kiếm, lọc theo tag hoặc danh mục, bảng xếp hạng và trang chi tiết;
- **Trải nghiệm đọc:** đọc chương theo URL thân thiện, lưu lịch sử, lưu

- chương đã đọc và tùy chọn hiển thị;
- **Tương tác:** theo dõi, đánh giá sao, bình luận, trả lời, bình chọn và báo cáo vi phạm;
 - **Đăng nội dung:** tạo truyện, import tệp văn bản/EPUB, quản lý chương, tag và cộng tác viên;
 - **Kiểm duyệt:** hàng chờ truyện, yêu cầu sửa, từ chối/duyet, xử lý bình luận, hồ sơ và báo cáo;
 - **Quản trị:** quản lý user, role, trạng thái tài khoản, truyện, từ khóa, báo cáo và audit log;
 - **Tính năng hỗ trợ:** tóm tắt chương và gợi ý truyện bằng AI, notification, worker và hàng đợi tác vụ nền.

1.5.2. Ngoài phạm vi hoặc chưa đủ minh chứng

Các nội dung ngoài phạm vi hoặc chưa đủ bằng chứng gồm:

- thanh toán, gói thuê bao, quảng cáo, bản quyền nội dung và chia sẻ doanh thu;
- ứng dụng di động native hoặc chế độ đọc ngoại tuyến;
- kiểm thử tải, kiểm thử xâm nhập và chỉ số SLA trên môi trường production;
- khả năng chịu lỗi của Redis/worker và tính đúng đắn của hàng đợi khi hệ thống khởi động lại;
- liên kết website production, video demo và dashboard giám sát thực tế;
- số liệu người dùng thật, dữ liệu vận hành thật hoặc kết quả kinh doanh.

Các liên kết và ảnh chụp được giữ trống để nhóm bổ sung bằng chứng đúng phiên bản nộp.

1.6. Yêu cầu phi chức năng

Bảng 1.3: Yêu cầu phi chức năng và giải pháp đáp ứng

Thuộc tính	Yêu cầu	Giải pháp/giới hạn hiện tại
Bảo mật	Route riêng tư phải được xác thực và phân quyền	JWT, Helmet, CORS, rate limiter, RBAC và audit middleware; chưa có kết quả penetration test.
Hiệu năng	Tránh response lớn và kết nối DB lặp lại	PostgreSQL pool, pagination, index, API danh sách chương không trả content; bundle frontend còn cảnh báo kích thước.
Khả dụng	Giao diện phản hồi trên các trạng thái tải/lỗi	Skeleton, optimistic UI có rollback ở một số tương tác; chưa có số liệu uptime.
Dễ bảo trì	Tách trách nhiệm và có quy ước source	Frontend theo các nhóm pages, components, services, contexts; backend theo routes, controllers, models, services, middleware và workers.
Khả năng mở rộng	Tác vụ dài không chặn request chính	Redis/BullMQ và process worker riêng; cần kiểm thử tích hợp với hạ tầng thật.
Truy vết	Thao tác quản trị quan trọng có lịch sử	Audit log lưu actor, action, entity và details đã làm sạch dữ liệu nhạy cảm.
Tương thích	Hoạt động như SPA trên hosting tĩnh	Vercel rewrite về index.html; API URL cấu hình qua biến môi trường.

1.7. Phương pháp thực hiện

Nhóm thực hiện dự án theo các bước: phân tích yêu cầu, xác định tác nhân và luồng nghiệp vụ, thiết kế kiến trúc, xây dựng frontend–backend–database, kiểm thử và hoàn thiện tài liệu. Nội dung báo cáo được đối chiếu với:

- đặc tả yêu cầu và tài liệu kiến trúc trong thư mục docs;
- mã nguồn frontend, backend, schema và cấu hình triển khai;
- kết quả kiểm thử và build production tại thời điểm rà soát;
- ảnh chụp đúng phiên bản source do nhóm bổ sung.

Khi tài liệu và mã nguồn khác nhau, báo cáo ưu tiên hành vi được thể hiện trong mã nguồn và kết quả kiểm thử. Các kiểm thử dùng mock không thay thế cho kiểm thử tích hợp với PostgreSQL, Redis và dịch vụ bên ngoài.

1.8. Kết luận chương

Chương 1 đã trình bày bối cảnh, bài toán, mục tiêu, đối tượng sử dụng, phạm vi và yêu cầu phi chức năng của CMC Truyện. Trên cơ sở đó, Chương 2 phân tích chi tiết luồng hoạt động theo từng vai trò.

CHƯƠNG 2. PHÂN TÍCH LUỒNG HOẠT ĐỘNG

2.1. Tổng quan tác nhân và nguyên tắc phân quyền

Hệ thống sử dụng kiểm soát truy cập theo vai trò. Backend là lớp quyết định quyền cuối cùng thông qua `authenticateToken` và `authorizeRole`; route guard ở frontend chủ yếu cải thiện trải nghiệm bằng cách không hiển thị màn hình mà người dùng không được phép truy cập. Không được xem frontend guard là biện pháp bảo mật duy nhất, vì người dùng vẫn có thể tự gửi HTTP request đến API.

Bảng 2.1: Ma trận quyền theo vai trò được thể hiện trong source

Nhóm hành động	Guest	User	Uploader	Moderator	Admin
Xem truyện/chương công khai	Có	Có	Có	Có	Có
Tìm kiếm, bảng xếp hạng	Có	Có	Có	Có	Có
Lưu lịch sử, theo dõi, bình luận	Không	Có	Có	Có	Có
Tạo truyện/chương	Không	Không	Có	Không	Hạn chế*
Quản lý truyện được phép	Không	Không	Có	Không	Có
Duyệt truyện chờ	Không	Không	Không	Có	Có
Xử lý báo cáo/bình luận	Không	Không	Không	Có	Có
Quản lý vai trò/tài khoản/từ khóa	Không	Không	Không	Không	Có
Xem audit log	Không	Không	Không	Có**	Có

* Route tạo truyện hiện yêu cầu đúng role Uploader, trong khi một số route sửa/xóa cho phép Uploader hoặc Admin. Đây là khác biệt cần giải thích khi demo.

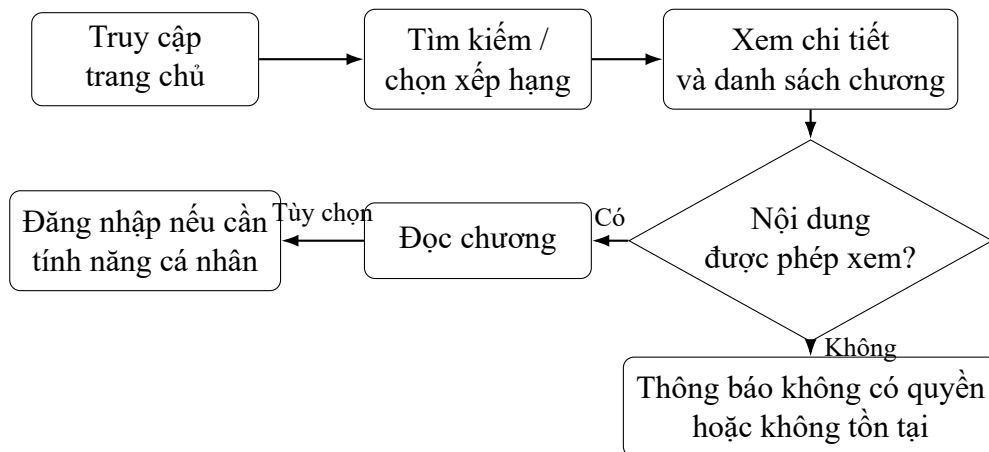
** Moderator chỉ được xem phạm vi audit phù hợp theo logic backend/test, không mặc nhiên có toàn quyền như Admin.

Ma trận được rút ra trực tiếp từ router và middleware: tạo truyện dành cho Uploader, duyệt truyện dành cho Moderator/Admin, còn khu vực `/api/admin` chỉ dành cho Admin.

2.2. Luồng của khách và người dùng

2.2.1. Khám phá và đọc truyện khi chưa đăng nhập

Guest có thể duyệt, tìm kiếm, xem chi tiết và đọc nội dung công khai. Route chi tiết dùng optionalAuth để bổ sung dữ liệu cá nhân hóa khi request có token hợp lệ.

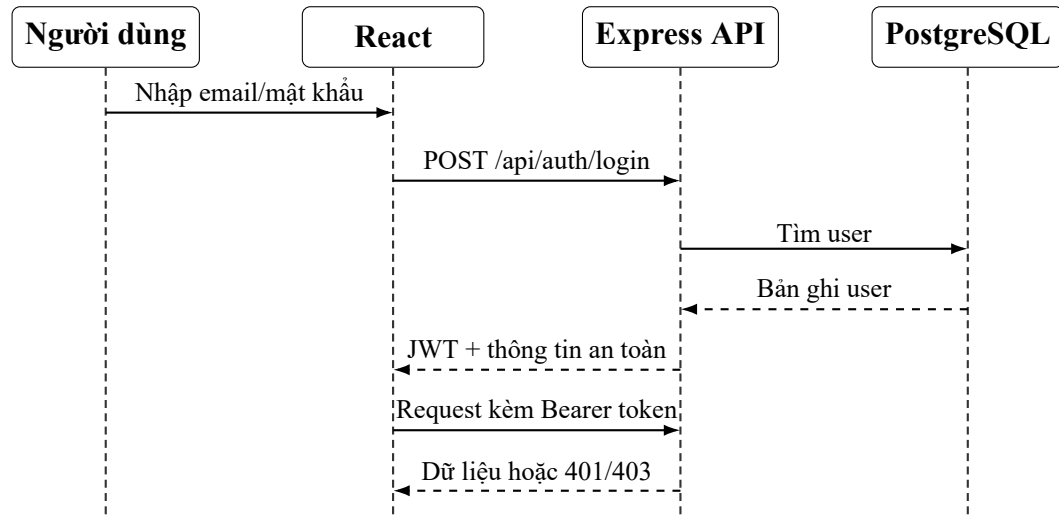


Hình 2.1: Luồng khám phá và đọc truyện của Guest

Giao diện cần phân biệt trạng thái đang tải, không có dữ liệu, không có quyền và lỗi mạng.

2.2.2. Đăng ký, đăng nhập và duy trì phiên

Frontend gửi thông tin đăng nhập đến POST/api/auth/login; backend kiểm tra mật khẩu và trả JWT. Token được lưu với key cmc_token, gắn vào request bằng Axios interceptor và xác minh tại middleware backend.



Hình 2.2: Trình tự xác thực và sử dụng JWT

Token không hợp lệ nhận mã 401 và phiên phía client bị xóa; mã 403 cho biết người dùng đã đăng nhập nhưng không đủ quyền. Luồng quên mật khẩu gồm yêu cầu OTP, xác minh và đặt mật khẩu mới; việc gửi email thật cần kiểm thử tích hợp riêng.

2.2.3. Theo dõi, lịch sử và tương tác

Sau khi đăng nhập, người dùng có thể theo dõi, đánh giá và bình luận. Một số thao tác dùng optimistic update và phải rollback nếu API thất bại.

Khi đọc chương, hệ thống gửi tiến độ đến POST/api/reading-history. Backend dùng định danh user từ JWT thay vì tin vào user id do client tự khai báo. Các endpoint lịch sử và chương đã đọc đều yêu cầu xác thực. Luồng điển hình gồm:

1. người dùng mở trang chi tiết và chọn chương;
2. frontend lấy nội dung chương theo slug và số chương;
3. trình đọc hiển thị nội dung, tiến độ cuộn và tùy chọn đọc;
4. sau một mốc phù hợp, frontend lưu chương/vị trí/thời gian đọc;
5. lần truy cập sau, lịch sử cho phép quay lại truyện và chương gần nhất;
6. nếu request lưu thất bại, giao diện không được khẳng định đã đồng bộ.

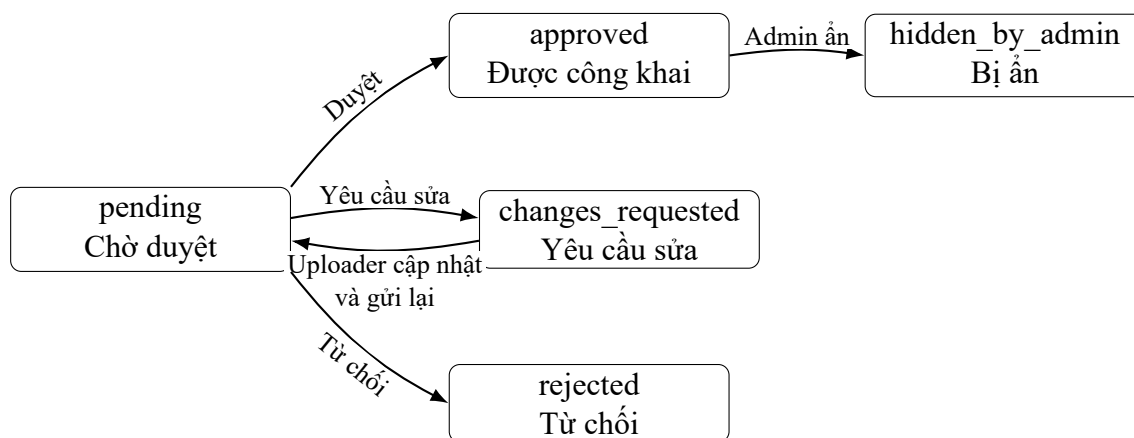
Report API áp dụng rate limiter; Moderator/Admin xử lý báo cáo và lưu kết quả giải quyết.

2.3. Luồng của Uploader

2.3.1. Tạo truyện và nhập nội dung

Uploader vào Dashboard, nhập metadata hoặc import tệp. Backend nhận thông tin, tạo truyện với `moderation_status='pending'` và mặc định chưa công khai. Khi import, hệ thống có bước preview để người dùng kiểm tra cách tách chương trước khi ghi dữ liệu. Source hỗ trợ tệp văn bản và EPUB; test kiểm tra thứ tự spine, tiêu đề chương, trường hợp không có tệp và fallback từ tên file.

Luồng tạo nội dung được thiết kế theo nguyên tắc “người đăng không tự duyệt nội dung của chính mình”. Test backend xác nhận Uploader không thể thêm chương trước khi truyện được Moderator duyệt và không thể dùng thao tác visibility để tự xuất bản. Điều này giảm rủi ro bỏ qua quy trình kiểm duyệt.



Hình 2.3: Vòng đời kiểm duyệt của truyện

2.3.2. Quản lý chương và cộng tác viên

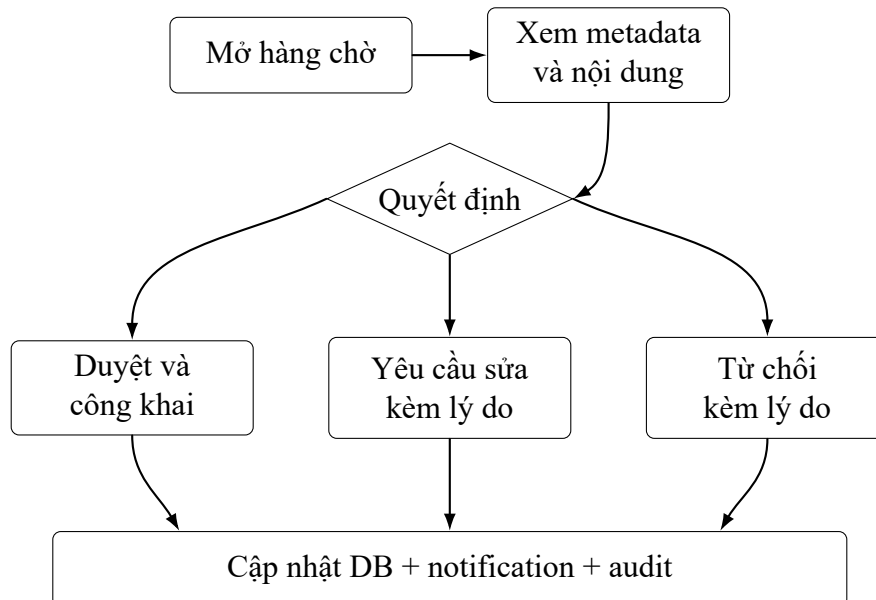
Sau khi truyện được duyệt, Uploader có thể thêm, sửa, xóa hoặc import chương theo quyền sở hữu/cộng tác. Backend phải kiểm tra quan hệ với truyện, không chỉ kiểm tra role chung. Ràng buộc `UNIQUE(story_id,chapter_number)` ngăn trùng số chương; controller chuyển xung đột dữ liệu thành phản hồi phù hợp thay vì trả lỗi máy chủ chung chung.

Chức năng cộng tác viên cho phép chia sẻ quyền làm nội dung trên một truyện. Đây là luồng nhạy cảm vì việc thêm hoặc xóa cộng tác viên thay đổi quyền truy cập gián tiếp. Khi hoàn thiện sản phẩm, cần kiểm tra đầy đủ: chỉ owner/admin được quản lý cộng tác viên, không tự thêm trùng, không cấp quyền vượt quá phạm vi truyện và phải có audit nếu yêu cầu vận hành đặt ra.

2.4. Luồng của Moderator

Moderator đăng nhập vào layout riêng, xem dashboard và hàng chờ truyện. Với mỗi truyện pending, người kiểm duyệt xem metadata và phần nội dung cần thiết trước khi chọn một trong ba hướng: duyệt, yêu cầu sửa kèm lý do, hoặc từ chối. Khi trạng thái thay đổi, backend cập nhật `reviewed_by`, `reviewed_at`, ghi audit và tạo notification phù hợp cho Uploader.

Với bình luận và hồ sơ bị báo cáo, Moderator dùng bộ lọc để tìm trường hợp cần xử lý, xem ngữ cảnh và cập nhật trạng thái. Test hiện có kiểm tra việc lọc ở server, số đếm trạng thái toàn cục và một số thao tác xử lý avatar. Điểm quan trọng là người kiểm duyệt cần thấy đủ ngữ cảnh để ra quyết định, nhưng không được nhận dữ liệu nhạy cảm không liên quan.



Hình 2.4: Luồng Moderator xử lý truyện chờ duyệt

2.5. Luồng của Admin

Admin có phạm vi vận hành rộng nhất nhưng vẫn đi qua xác thực, RBAC và audit. Các nhóm công việc chính gồm:

- xem thống kê tổng quan và danh sách truyện;
- tìm kiếm user, cập nhật role hoặc trạng thái hoạt động;
- ẩn/hiện truyện, xóa bình luận theo quy định;
- quản lý danh sách từ khóa và tier kiểm duyệt;
- xem, phân loại và giải quyết báo cáo;
- truy vấn audit log để đối chiếu hành động quản trị;
- thực hiện các chức năng moderator khi cần.

Bảng 2.2: Luồng quản trị chính và dấu vết cần lưu

Thao tác	Kiểm tra trước khi thực hiện	Dấu vết sau thao tác
Đổi role user	Admin hợp lệ; role đích thuộc tập cho phép; tránh tự khóa quyền quản trị cuối cùng	Actor, user bị tác động, role cũ/mới, thời điểm.
Khóa/mở tài khoản	Xác định đúng user và lý do vận hành	Actor, trạng thái cũ/mới, user bị tác động.
Ẩn/hiện truyện	Kiểm tra truyện tồn tại và quyền quản trị	Action, story id, trạng thái hiển thị.
Quản lý từ khóa	Keyword/tier hợp lệ; tránh bản ghi trùng	Tạo/sửa/xóa từ khóa và tier; không ghi dữ liệu nhạy cảm.
Xử lý báo cáo	Report còn tồn tại; action phù hợp target	Trạng thái, action, note, resolver và thời điểm.
Xóa bình luận	Xác định vi phạm và phạm vi tác động	Actor, comment id và kết quả HTTP thành công.

Audit middleware chỉ ghi mutation quản trị thành công và làm sạch trường nhạy cảm trong request. Test kiểm tra password/credential lồng nhau không bị đưa vào details. Đây là yêu cầu bảo mật quan trọng: audit log phục vụ truy vết nhưng không được trở thành nơi rò rỉ bí mật.

2.6. Luồng báo cáo vi phạm

Luồng report kết nối người dùng, moderator và admin. Người dùng đăng nhập chọn đối tượng cần báo cáo, lý do và mô tả. Rate limiter hạn chế việc gửi lặp; controller xác thực target và tạo report. Moderator/Admin lọc theo trạng thái, xem ngữ cảnh, chọn hành động giải quyết rồi cập nhật trạng thái cùng ghi chú.

Các test backend xác nhận một số quy tắc: tạo report truyện hợp lệ; gắn báo cáo avatar với tác giả bình luận liên quan; ngăn spam trong một giờ; lọc report theo trạng thái; trả 404 khi report không tồn tại; không dùng action của chapter cho report comment; và lưu dữ liệu giải quyết. Những test này chứng minh logic nhánh quan trọng, nhưng vẫn cần demo trực quan để xác nhận UI truyền đúng payload và hiển thị phản hồi dễ hiểu.

2.7. Truy vết luồng đến endpoint và source

Bảng 2.3: Truy vết luồng nghiệp vụ đến API và module hiện thực

Luồng	Endpoint tiêu biểu	Quyền	Module chính
Đăng nhập	POST/api/auth/login	Public	authRoutes, authController
Xem truyện	GET/api/stories/by-slug/:slug	Optional auth	storyRoutes, storyController
Đọc chương	GET/api/stories/by-slug/:storySlug/chapters/:chapterNumber	Optional auth	chapterController
Lưu tiến độ	POST/api/reading-history	Authenticated	readingHistory controller
Theo dõi	POST/DELETE/api/follows/:storyId	Authenticated	followRoutes, model liên quan
Tạo truyện	POST/api/stories	Uploader	storyController. createStory
Import truyện	POST/api/stories/import-file	Uploader	upload middleware, import service
Duyệt truyện	PATCH/api/moderator/pending-stories/:id/approve	Moderator/ Admin	moderator controller, audit middleware
Xử lý report	PATCH/api/reports/:id/process	Moderator/ Admin	report controller, audit middleware
Đổi role user	PATCH/api/admin/users/:id/role	Admin	admin controller, audit middleware
Quản lý từ khóa	/api/admin/bad-words	Admin	bad word controller/model
Xem audit	GET/api/audit-logs	Admin/ Moderator theo phạm vi	audit log route, controller, model

2.8. Quy tắc xử lý lỗi và an toàn luồng

Một luồng được xem là hoàn chỉnh khi có cả đường thành công và đường lỗi. Các nguyên tắc được áp dụng hoặc cần duy trì gồm:

- **400** cho dữ liệu đầu vào không hợp lệ, kèm thông báo đủ để người dùng sửa;
- **401** khi thiếu hoặc sai phiên xác thực;
- **403** khi đã xác thực nhưng không đủ quyền;
- **404** khi tài nguyên không tồn tại hoặc không nên lộ sự tồn tại;
- **409** cho xung đột nghiệp vụ như trùng số chương;
- **429** khi vượt giới hạn request/report;
- **500** chỉ cho lỗi ngoài dự kiến, không trả stack trace hoặc secret ra client;
- mutation nhiều bước cần transaction cùng một DB client để bảo đảm roll-back nhất quán;
- optimistic UI phải hoàn tác khi API thất bại;
- mọi quyết định quyền phải được lặp lại ở backend dù frontend đã ẩn nút.

2.9. Kết luận chương

Chương 2 đã mô tả luồng của Guest, User, Uploader, Moderator và Admin, gồm trạng thái, ngoại lệ và điểm kiểm soát quyền. Các luồng được truy vết tới endpoint để làm cơ sở cho phân triển khai.

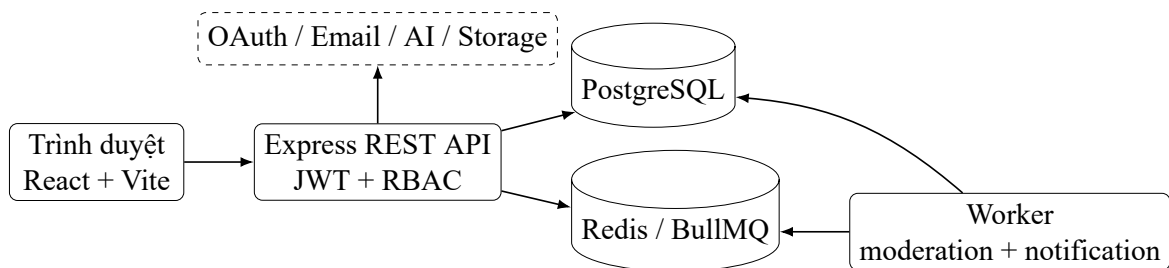
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG

3.1. Kiến trúc phần mềm tổng thể

3.1.1. Lựa chọn mô hình kiến trúc

Mã nguồn hiện thực kiến trúc frontend/backend tách rời. Frontend là ứng dụng đơn trang (SPA) xây dựng bằng React và Vite; trình duyệt trao đổi với REST API Express qua HTTP/JSON. Backend xác thực JWT, điều phối nghiệp vụ và truy vấn PostgreSQL qua connection pool. Redis/BullMQ phục vụ hàng đợi; worker chạy ở tiến trình riêng để xử lý kiểm duyệt và thông báo. Một số dịch vụ bên ngoài được tích hợp cho OAuth, email, lưu trữ và AI [2, 3, 4].

Frontend đảm nhiệm giao diện, tương tác và điều hướng phía client. Backend được tổ chức theo các lớp Route–Controller–Model/Service; cách tách này giúp định tuyến, xử lý nghiệp vụ và truy cập dữ liệu có trách nhiệm rõ ràng [5].



Hình 3.1: Kiến trúc logic của hệ thống theo source hiện tại

Trong request thông thường, React gọi API; middleware kiểm tra CORS, parse body, xác thực và phân quyền; controller validate/điều phối; model hoặc service truy cập dữ liệu; error handler chuẩn hóa phản hồi. Với tác vụ nền, API ghi job vào queue và trả phản hồi sớm; worker nhận job, xử lý rồi cập nhật PostgreSQL hoặc tạo notification. Việc tách process giúp tránh chặn event loop của web server, nhưng chỉ phát huy tác dụng khi Redis và worker được cấu hình/giám sát đúng.

3.1.2. Công nghệ và trách nhiệm

Bảng 3.1: Stack công nghệ được xác nhận từ package và cấu hình dự án

Lớp	Công nghệ	Trách nhiệm
Frontend	React 18, React Router 6, Vite 5	Component UI, route SPA, state phía client và build tài nguyên tĩnh.
Giao tiếp	Axios	Base URL, JSON, gắn JWT và xử lý lỗi 401/403 tập trung.
Trình bày	CSS, Bootstrap 5, Tailwind CSS trong dependencies	Bố cục, responsive, trạng thái sáng/tối và trải nghiệm đọc.
Backend	Node.js, Express 4	REST API, middleware pipeline, controller và tích hợp dịch vụ.
Xác thực	JWT, bcryptjs, Google Auth Library	Phiên không trạng thái, băm mật khẩu và Google OAuth.
Cơ sở dữ liệu	PostgreSQL, thư viện pg	Dữ liệu quan hệ, ràng buộc, index và connection pool.
Tác vụ nền	Redis, BullMQ	Queue cho moderation/notification và worker riêng.
Lưu trữ/tích hợp	Supabase, Resend, Groq/Gemini	Ảnh, email OTP và tính năng AI qua backend.
Kiểm thử	Jest, Supertest, Vitest, Testing Library, Playwright	Unit/integration mức module và kịch bản E2E theo cấu hình.
Triển khai	Vercel-compatible SPA, Render web service/worker	Hosting frontend tĩnh, API và process worker.

Các phiên bản trong Bảng 3.1 được lấy từ frontend/package.json, backend/package.json và lockfile tại thời điểm rà soát. Báo cáo không khẳng định mọi dependency đã được sử dụng đồng đều chỉ vì xuất hiện trong package; vai trò cụ thể được đối chiếu thêm với import và cấu hình source.

3.2. Triển khai frontend

3.2.1. Cấu trúc thư mục

Entry point frontend là frontend/src/main.jsx, sau đó render App.jsx. Source được tổ chức theo pages, components, layouts, contexts, services, utils, styles và data. Tại thời điểm thống kê, thư mục pages có 28 file chính và components có 31 file chính; số lượng này là ảnh chụp trạng thái codebase, không phải chỉ tiêu chất lượng tự thân.

```
frontend/
├── src/
│   ├── pages/                (28 tệp chính)
│   ├── components/          (31 tệp chính)
│   ├── layouts/
│   ├── contexts/
│   ├── services/
│   │   └── api.js
│   ├── utils/
│   ├── styles/
│   ├── data/
│   ├── App.jsx
│   └── main.jsx
```

Hình 3.2: Cấu trúc thư mục frontend

Bảng 3.2: Trách nhiệm các thư mục frontend chính

Thư mục/tệp	Trách nhiệm
App.jsx	Khai báo route, layout công khai, Admin, Moderator và route guard.
pages/	Component cấp trang cho Home, Search, Reader, Account, Dashboard và quản trị.
components/	Thành phần tái sử dụng như Navbar, StoryCard, CommentSection, FollowButton.
layouts/	Khung giao diện riêng cho Admin và Moderator.
contexts/	Trạng thái xác thực và theme chia sẻ trong cây component.
services/api.js	Axios client và facade endpoint theo domain.
utils/	Validation, slug, số chương và chuẩn hóa lỗi.
styles/	CSS chung, dark mode, reader và màn hình quản lý.

3.2.2. Điều hướng và bảo vệ route

App.jsx chia route thành ba nhóm. Nhóm user/public dùng Navbar và Footer; nhóm Admin dùng layout riêng (AdminLayout); nhóm Moderator dùng layout riêng (ModeratorLayout). ProtectedRoute yêu cầu đăng nhập, còn RoleProtectedRoute kiểm tra role trước khi render Dashboard hoặc layout quản trị. Cấu trúc này giảm lặp và giúp sidebar/menu nhất quán trong từng khu vực.

```

frontend/src/App.jsx
119     <Route
120       path="/dashboard"
121       element={
122         <RoleProtectedRoute allowedRoles={['Uploader', 'Admin']}>
123           <DashboardPage />
124         </RoleProtectedRoute>
125       }
126     />
131     {/* ADMIN LAYOUT RIÊNG */}
134     <RoleProtectedRoute allowedRoles={['Admin']}>
135       <AdminLayout />
136     </RoleProtectedRoute>
149     {/* MODERATOR LAYOUT RIÊNG */}
152     <RoleProtectedRoute allowedRoles={['Moderator', 'Admin']}>
153       <ModeratorLayout />
154     </RoleProtectedRoute>

```

```

frontend/src/components/ProtectedRoute.jsx
5  function ProtectedRoute({ children }) {
6    const { isAuthenticated } = useAuth();
7    const location = useLocation();
9    if (!isAuthenticated) {
10     return <Navigate to="/login" replace state={{ from: location }} />;
11   }
13   return children;

```

```

frontend/src/components/RoleProtectedRoute.jsx
4  function RoleProtectedRoute({ allowedRoles = [], children }) {
6    const { isAuthenticated, user, loading } = useAuth();
7    const location = useLocation();
16   if (!isAuthenticated) {
17     return <Navigate to="/login" replace state={{ from: location }} />;
18   }
21   const userRole = user?.role?.toLowerCase();
22   const allowed = allowedRoles.map(r => r.toLowerCase());
24   if (allowedRoles.length > 0 && !allowed.includes(userRole)) {
25     return <Navigate to="/" replace />;
26   }
28   return children;
29 }

```

Hình 3.3: Đoạn code chính về route và route guard ở frontend

Route frontend không thay thế quyền backend. Nếu chỉ ẩn menu Admin nhưng API không có middleware, người dùng vẫn có thể gọi endpoint bằng công cụ khác. Source hiện đã đặt RBAC ở backend; frontend guard được xem là lớp trải nghiệm và phòng lỗi điều hướng.

3.2.3. Lớp gọi API và quản lý phiên

services/api.js tạo một Axios instance có base URL từ biến môi trường, fallback về local khi phát triển. Request interceptor đọc cmc_token và gắn Bearer token; response interceptor xử lý 401/403, xóa phiên và điều hướng về login trong các trường hợp phù hợp. Các endpoint được gom theo domain như auth, stories, chapters, comments, follows, reports, admin và moderator.

Ưu điểm của facade tập trung là component không cần lặp base URL, header và cấu trúc request. Hạn chế là file có thể lớn khi mọi domain cùng nằm trong một object; về lâu dài có thể tách thành các module authApi, storyApi, adminApi nhưng vẫn dùng chung Axios client.

Base URL theo biến môi trường	Dòng 15–21
<pre>15 function getBaseUrl() { 16 return (17 import.meta.env.VITE_API_URL // Vite environment variable 18 import.meta.env.REACT_APP_API_URL // Create React App (legacy) 19 'http://localhost:5000/api' 20); 21 }</pre>	
Axios instance dùng chung	Dòng 37–43
<pre>37 const apiClient = axios.create({ 38 baseUrl: getBaseUrl(), 39 headers: { 40 'Content-Type': 'application/json', 41 Accept: 'application/json', 42 }, 43 });</pre>	
Request interceptor — tự động gắn Bearer token	Dòng 50–64
<pre>50 apiClient.interceptors.request.use((config) => { 51 try { 52 const token = localStorage.getItem('cmc_token'); 53 if (token && token !== 'null') { 54 config.headers['Authorization'] = `Bearer \${token}`; 55 } 56 } catch (err) { 57 // localStorage may be unavailable in some environments (private mode, strict policies). 58 // Silently ignore so the app doesn't spam the console or throw runtime errors. 59 } 60 return config; 61 }, (error) => { 62 return Promise.reject(error); 63 });</pre>	
Response interceptor — xử lý lỗi 401/403	Dòng 76–91
<pre>76 apiClient.interceptors.response.use(77 (response) => response, // Nếu thành công, trả về nguyên response 78 (error) => { 79 if ((error?.response?.status === 401 error?.response?.status === 403) && typeof window !== 'undefined') { 80 const path = window.location.pathname; 81 82 // Chỉ redirect nếu KHÔNG đang ở các trang public (login, register, home) 83 // Tránh redirect loop: login page nhận 401/403 → redirect về login 84 if (!path.startsWith('/login') && !path.startsWith('/register') && path !== '/') { 85 clearAuthStorage(); 86 window.location.href = '/login'; 87 } 88 } 89 return Promise.reject(error); 90 } 91);</pre>	

Hình 3.4: Đoạn code chính về Axios interceptor ở frontend

3.2.4. Trạng thái giao diện và trải nghiệm

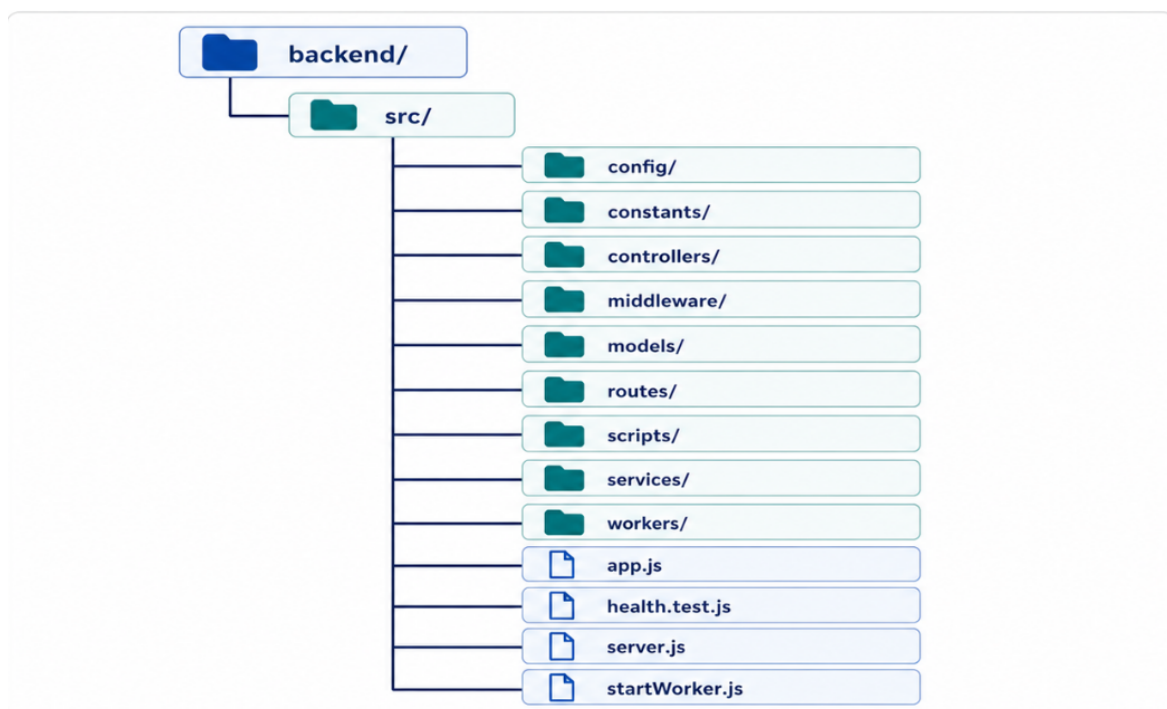
Source có skeleton screen cho trạng thái tải và optimistic UI kèm rollback ở những tương tác phù hợp như follow, rating hoặc vote. Theme và tùy chọn đọc được tách thành context/panel riêng. Khi đánh giá frontend, cần kiểm tra đủ bốn trạng thái: loading, success có dữ liệu, success rỗng và error. Một giao diện đẹp ở trạng thái thành công nhưng vỡ khi API chậm vẫn chưa phải trải nghiệm hoàn chỉnh.

Về khả năng truy cập, báo cáo chưa có kết quả audit tự động. Nhóm cần kiểm tra bằng bàn phím, focus, label của form, contrast ở dark mode, alt text ảnh bìa và thông báo lỗi có thể đọc được. Đây là hạng mục cần bổ sung bằng minh chứng thật nếu rubric của giảng viên yêu cầu.

3.3. Triển khai backend

3.3.1. Cấu trúc phân lớp

Backend bắt đầu ở src/server.js, tạo HTTP server từ app.js. Router chỉ định URL và middleware; controller validate, điều phối và tạo HTTP response; model/service thực hiện query hoặc tích hợp bên ngoài; worker xử lý job. Thống kê source chính cho thấy 16 file route, 17 controller, 17 model, 9 service và 6 middleware, không tính file test.



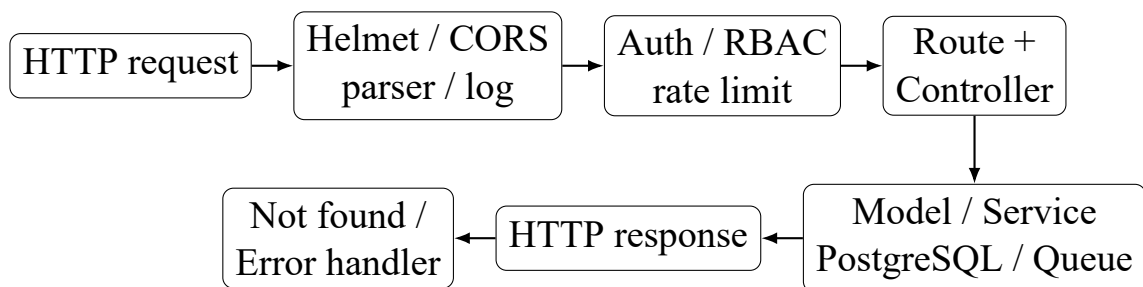
Hình 3.5: Cấu trúc thư mục backend

Bảng 3.3: Phân lớp backend và nguyên tắc phụ thuộc

Lớp	Trách nhiệm	Nguyên tắc
Routes	URL, method, auth, role, upload, rate limit, audit	Không chứa truy vấn SQL dài hoặc nghiệp vụ phức tạp.
Controllers	Validate request, điều phối model/service, response	Không tin user id/role do client tự gửi khi đã có JWT.
Models	Truy vấn và ánh xạ dữ liệu	Parameterized query; transaction dùng cùng DB client.
Services	Nghiệp vụ/tích hợp dùng lại	Che chi tiết AI, email, storage, queue khỏi controller.
Middleware	Xử lý cắt ngang	Auth trước role; audit sau khi biết response thành công.
Workers	Consumer job nền	Chạy process riêng, idempotent khi có thể.
Config	Biến môi trường và client hạ tầng	Secret chỉ ở backend; không commit .env.

3.3.2. Pipeline request

app.js lần lượt cấu hình Helmet, CORS, log HTTP, các parser JSON và URL-encoded, static cover, health check, 16 nhóm router rồi not-found và error handler. Thứ tự middleware quan trọng: error handler phải ở cuối; role middleware phải sau auth; audit cần quan sát kết quả response nhưng không được ghi dữ liệu nhạy cảm.



Hình 3.6: Pipeline xử lý request ở backend

```
backend/src/app.js
31 const app = express();
32 // Security & Middleware
33 app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
34 app.use(
35   cors({
36     credentials: true,
37   })
38 );
39 app.use(morgan(env.isDevelopment ? 'dev' : 'combined'));
40 app.use(express.json({ limit: '10mb' }));
41 app.use(express.urlencoded({ extended: true }));
42 app.use('/uploads/covers', express.static(uploadDir));
43 // Health Check
44 app.get('/health', (req, res) => {
45   res.status(200).json({
46     success: true,
47     message: 'CMC Truyen backend is running',
48     environment: env.NODE_ENV,
49   });
50 });
51 // Route đăng ký
52 app.use('/api/auth', authRoutes);
53 app.use('/api/stories', storyRoutes);
54 app.use('/api/reading-history', readingHistoryRoutes);
55 app.use('/api/chapters', chapterRoutes);
56 app.use('/api/ai', aiRoutes);
57 app.use('/api/comments', commentRoutes);
58 app.use('/api/follows', followRoutes);
59 app.use('/api/notifications', notificationRoutes);
60 app.use('/api/preferences', preferencesRoutes);
61 app.use('/api/upload', uploadRoutes);
62 app.use('/api/admin', adminRoutes);
63 app.use('/api/moderator', moderatorRoutes);
64 app.use('/api/tags', tagRoutes);
65 app.use('/api/reports', reportRoutes);
66 app.use('/api/rankings', rankingRoutes);
67 app.use('/api/audit-logs', auditLogRoutes);
68 // Error Handling
69 app.use(notFoundHandler);
70 app.use(errorHandler);
71 module.exports = app;
```

Hình 3.7: Đoạn code chính cấu hình middleware và mount router

3.3.3. Xác thực, phân quyền và audit

authenticateToken đọc header Authorization, yêu cầu định dạng Bearer, xác minh JWT bằng secret từ environment và gán payload vào req.user. optionalAuth không từ chối Guest; token lỗi được xem như không có phiên ở các route công khai. authorizeRole chuẩn hóa chữ thường để tránh sai khác giữa Admin và admin. Cấu trúc và cách kiểm tra token được đối chiếu với đặc tả JWT [7].

Audit middleware bao quanh các thao tác làm thay đổi dữ liệu quản trị như đổi vai trò, đổi trạng thái người dùng, duyệt truyện, xử lý báo cáo và quản lý từ khóa. Kiểm thử xác nhận hệ thống chỉ ghi hành động thành công, giới hạn phạm vi Moderator và loại dữ liệu mật khẩu/credential khỏi nhật ký. Các biện pháp xác thực, phân quyền, kiểm soát đầu vào và ghi log được rà soát theo nhóm rủi ro ứng dụng web phổ biến của OWASP [10].

```

backend/src/middleware/authMiddleware.js
16 function authenticateToken(req, res, next) {
17   const authHeader = req.headers.authorization;
22   if (!authHeader || !authHeader.startsWith('Bearer ')) {
23     return res.status(401).json({
24       success: false,
25       message: 'No token provided',
26     });
30   const token = authHeader.split(' ')[1];
32   try {
35     const decoded = jwt.verify(token, env.JWT_SECRET);
39     req.user = decoded;
40     return next();
41   } catch (error) {
43     return res.status(401).json({
44       success: false,
45       message: 'Invalid or expired token',
46     });
48   }
60 function optionalAuth(req, res, next) {
61   const authHeader = req.headers.authorization;
64   if (authHeader && authHeader.startsWith('Bearer ')) {
65     const token = authHeader.split(' ')[1];
66     try {
68       req.user = jwt.verify(token, env.JWT_SECRET);
69     } catch {
71       req.user = null;
73     } else {
75       req.user = null;
78       return next();
79     }
}

backend/src/middleware/roleMiddleware.js
5 function authorizeRole(...roles) {
6   return (req, res, next) => {
8     if (!req.user || !req.user.role) {
9       return res.status(403).json({
11        message: 'Access denied: No role information found',
12      });
17     const userRole = req.user.role.toLowerCase();
18     const allowedRoles = roles.map(role => role.toLowerCase());
21     if (!allowedRoles.includes(userRole)) {
22       return res.status(403).json({
24        message: 'Access denied: Insufficient permissions',
25      });
29     return next();
30   };
31 }

```

Hình 3.8: Đoạn code chính về xác thực JWT và RBAC

3.3.4. Tác vụ nền và dịch vụ AI

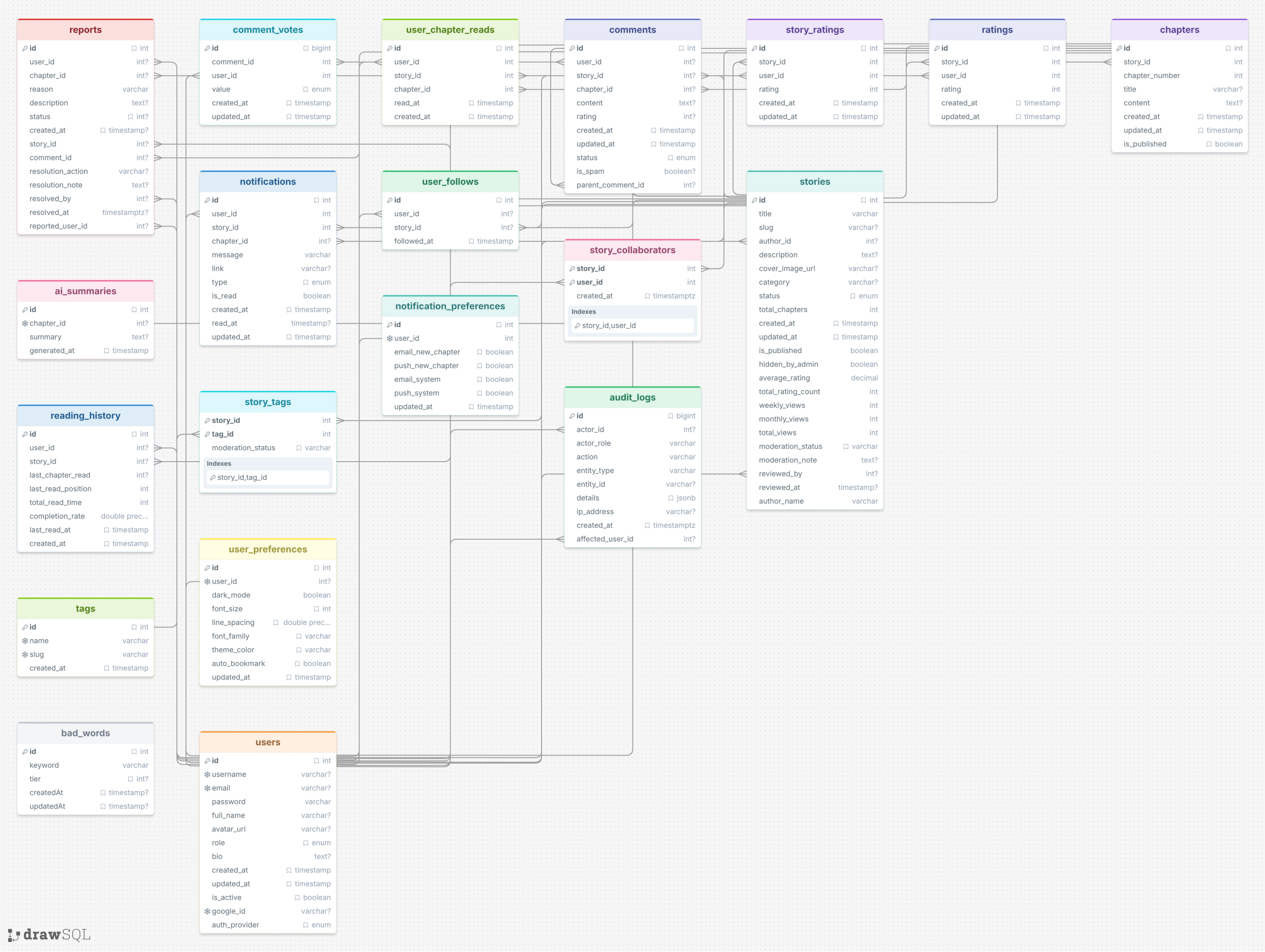
startWorker.js khởi động moderation worker và notification worker. Queue dùng REDIS_URL; API và worker phải trở cùng Redis [8, 9]. Với AI, backend ưu tiên Groq rồi fallback Gemini, có timeout và cache; tóm tắt chương được lưu ở ai_summaries. API key không đi xuống frontend vì mã phía client có thể được người dùng xem trực tiếp.

Tuy nhiên, fallback giữa hai mô hình có thể tạo kết quả khác nhau; hệ thống cần log provider, thời gian và lỗi ở mức đủ chẩn đoán nhưng không ghi prompt nhạy cảm. Với moderation tự động, quyết định có tác động lớn nên có cơ chế review của người vận hành thay vì coi output AI là phán quyết tuyệt đối.

3.4. Triển khai cơ sở dữ liệu

3.4.1. Mô hình dữ liệu

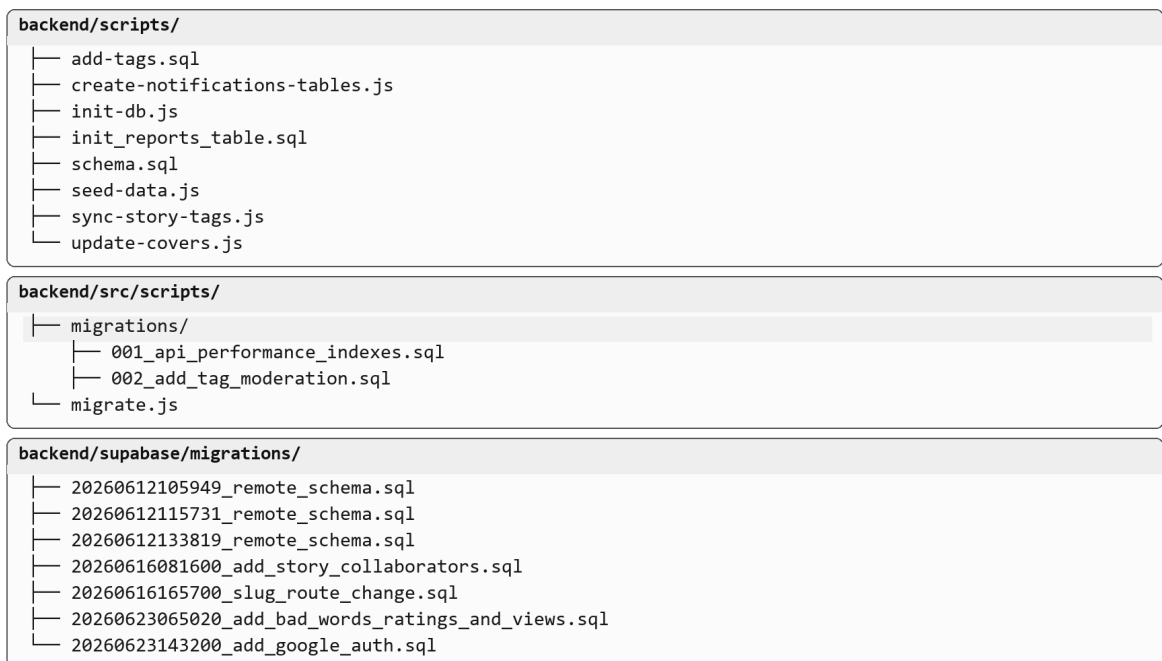
Schema chính backend/scripts/schema.sql định nghĩa các nhóm dữ liệu về tài khoản, truyện và chương, lịch sử đọc, tương tác, thông báo, kiểm duyệt và nhật ký quản trị. Các script riêng bổ sung tag, liên kết truyện–tag và báo cáo vi phạm. Thiết kế sử dụng khóa chính, khóa ngoại, UNIQUE và CHECK để đảm bảo tính toàn vẹn dữ liệu [6].



Hình 3.9: Lược đồ ERD của cơ sở dữ liệu CMC Truyện

Hình 3.9 thể hiện các quan hệ chính của hệ thống. `users`, `stories` và `chapters` là ba thực thể trung tâm; các bảng lịch sử đọc, theo dõi, đánh giá, bình luận và thông báo liên kết hành vi người dùng với nội dung. Nhóm bảng `reports`, `bad_words` và `audit_logs` phục vụ kiểm duyệt và truy vết.

ERD do nhóm cung cấp có đồng thời `ratings` và `story_ratings`, trong khi schema chính được rà soát sử dụng `ratings`. Trước khi nộp, nhóm cần xác nhận bảng đang được ứng dụng sử dụng, loại bỏ bảng dư hoặc giải thích rõ mục đích của từng bảng; đồng thời bảo đảm ERD, migration và cơ sở dữ liệu triển khai cùng một phiên bản.



Hình 3.10: Cấu trúc script, migration và schema cơ sở dữ liệu

01 • users
schema.sql • lines 21-29

PRIMARY KEY • UNIQUE • CHECK

```

21 CREATE TABLE IF NOT EXISTS users (
22     id SERIAL PRIMARY KEY,
23     username VARCHAR(100) UNIQUE,
24     email VARCHAR(255) UNIQUE,
25     password VARCHAR(255) NOT NULL,
26     full_name VARCHAR(255),
27     avatar_url VARCHAR(500),
28     role VARCHAR(50) NOT NULL DEFAULT 'User'
29     CHECK (role IN ('Admin', 'Uploader', 'User', 'Moderator')),

```

02 • chapters
schema.sql • lines 70-80

PRIMARY KEY • FOREIGN KEY • UNIQUE

```

70 CREATE TABLE IF NOT EXISTS chapters (
71     id SERIAL PRIMARY KEY,
72     story_id INTEGER NOT NULL REFERENCES stories(id) ON DELETE CASCADE,
73     chapter_number INTEGER NOT NULL,
74     title VARCHAR(255),
75     content TEXT,
76     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
77     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
78     is_published BOOLEAN NOT NULL DEFAULT true,
79     UNIQUE (story_id, chapter_number)
80 );

```

03 • story_collaborators
schema.sql • lines 199-204

COMPOSITE PRIMARY • FOREIGN KEYS

```

199 CREATE TABLE IF NOT EXISTS story_collaborators (
200     story_id INTEGER NOT NULL REFERENCES stories(id) ON DELETE CASCADE,
201     user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
202     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
203     PRIMARY KEY (story_id, user_id)
204 );

```

04 • indexes
schema.sql • lines 256-262

SIMPLE • PARTIAL • COMPOSITE

```

256 CREATE INDEX IF NOT EXISTS idx_stories_author_id ON stories(author_id);
257 CREATE INDEX IF NOT EXISTS idx_stories_slug ON stories(slug)
    WHERE slug IS NOT NULL;
260 CREATE INDEX IF NOT EXISTS idx_stories_category_published_created
    ON stories(category, created_at DESC) WHERE is_published = true;
262 CREATE INDEX IF NOT EXISTS idx_chapters_story_published_number
    ON chapters(story_id, chapter_number) WHERE is_published = true;

```

Hình 3.11: Đoạn SQL chính về bảng truyện, chương và ràng buộc

3.4.2. Connection pool, index và transaction

Backend cấu hình PostgreSQL Pool với tối đa 20 connection, idle timeout 30 giây và connection timeout 5 giây. Môi trường production có thể dùng DATABASE_URL; môi trường local có thể dùng các biến

host/port/name/user/password. Index tập trung vào slug, danh sách truyện công khai, số chương, trạng thái bình luận, khóa ngoại và audit. API danh sách chương không trả trường content, qua đó giảm kích thước phản hồi.

Khi một nghiệp vụ có nhiều bước ghi dữ liệu, tất cả query từ BEGIN đến COMMIT/ROLLBACK phải dùng cùng client lấy từ pool.connect(). Nếu trộn pool.query() với client trong một transaction, các query có thể chạy trên connection khác và rollback không còn bảo đảm tính nguyên tử.

3.4.3. Nợ kỹ thuật dữ liệu

Phần dữ liệu hiện có ba cơ chế cùng tồn tại: schema tổng, script bổ sung và một số DDL chạy khi module database kết nối. Cách tổ chức này làm nguồn sự thật bị phân tán và khó tái tạo một cơ sở dữ liệu mới. Bảng reports dùng script CREATETYPE và CREATETABLE chưa có IFNOTEXISTS; bad_words xuất hiện trên ERD nhưng chưa có DDL trong phần mã nguồn đã rà soát. Hướng hoàn thiện là:

- đưa mọi thay đổi vào migration đánh số và có thể chạy lặp an toàn;
- tạo migration chính thức cho bad_words, reports, tags/story_tags;
- xác nhận và xử lý sự trùng lặp giữa ratings và story_ratings;
- loại DDL biến đổi schema khỏi đường khởi động ứng dụng sau khi migration ổn định;
- bổ sung bảng theo dõi migration và kiểm thử khởi tạo DB trống;
- ghi rõ baseline schema tương ứng với commit dùng để nộp.

3.5. Cấu hình và triển khai

3.5.1. Biến môi trường

Các biến chính gồm URL frontend/backend, PostgreSQL, Redis, JWT, Google OAuth, Resend, Supabase và nhà cung cấp AI. File .env không được commit. Frontend chỉ nhận biến public như VITE_API_URL; service key, JWT secret và AI key phải ở backend.

3.5.2. Topology triển khai

render.yaml khai báo hai service Node: web service chạy backend và worker chạy src/startWorker.js. Frontend có vercel.json rewrite mọi route SPA về index.html. PostgreSQL và Redis không được khai báo trực tiếp trong file Render; chúng phải được cung cấp qua dịch vụ managed hoặc hạ tầng khác và cấu hình bằng environment.

Bảng 3.4: Nhóm biến môi trường và nguyên tắc bảo mật

Nhóm	Ví dụ tên biến	Nguyên tắc
Database	DATABASE_URL, DB_HOST, DB_NAME	Chỉ backend; production ưu tiên secret manager.
Auth	JWT_SECRET, GOOGLE_CLIENT_ID	Secret đủ mạnh; không dùng fallback production.
Queue	REDIS_URL	API và worker dùng cùng instance; bật TLS nếu nhà cung cấp yêu cầu.
Email/Storage	RESEND_API_KEY, SUPABASE_SERVICE_KEY	Service key không xuất hiện trong bundle frontend/log.
AI	GROQ_API_KEY, GEMINI_API_KEY	Chỉ backend; đặt timeout, quota và theo dõi lỗi.
Frontend	VITE_API_URL	Có thể công khai; trở đúng tiền tố /api.

Bảng 3.5: Liên kết bàn giao và minh chứng triển khai

Hạng mục	Liên kết
Website chạy thực tế	https://cmc-truyen.vercel.app
Kho source dùng để nộp	https://github.com/cmc-truyen/cmc-truyen-temp

3.6. Kiểm thử và kết quả xác minh

3.6.1. Kết quả thực chạy

Ngày 22/07/2026, bộ test mặc định và build frontend được chạy trên workspace cung cấp. Kết quả được ghi ở Bảng 3.6; đây là số liệu của lần chạy cụ thể, không phải con số giả định.

Bảng 3.6: Kết quả kiểm thử và build đã thực chạy

Hạng mục	Kết quả	Ghi chú quan sát
Backend baseline (Jest)	49/49 pass	12 test suite pass; có log Redis ECONNREFUSED và Jest force-exit do handle chưa đóng.
Frontend (Vitest)	23/23 pass	8 test file pass; có cảnh báo tùy chọn es-build/oxc deprecated từ toolchain.
Tổng test mặc định	72/72 pass	Lệnh root test hoàn thành exit code 0.
Frontend production build	Thành công	161 module transformed; build trong 2,75 giây ở lần chạy ghi nhận.
Bundle JavaScript	535,36 kB	Gzip 158,54 kB; Vite cảnh báo chunk lớn hơn 500 kB.
E2E Playwright	3/3 pass	100% pass (3 kịch bản E2E mock API kiểm tra luồng login, search và reader thành công).

Kết quả pass cho thấy logic đã được bảo vệ ở nhiều nhánh: health, auth, RBAC, audit, OTP, import file, ranking, moderation, report và workflow truyện. Tuy nhiên, log Redis thất bại cho thấy test chưa cô lập hoàn toàn queue hoặc còn resource mở. Cần đóng queue/connection trong teardown, mock Redis rõ ràng ở unit test và chạy một nhóm integration test với Redis thật. Cảnh báo bundle nên được xử lý bằng route-level lazy loading, dynamic import và chia vendor chunk; sau tối ưu phải đo lại thay vì chỉ tăng ngưỡng cảnh báo.

3.6.2. Kịch bản kiểm thử thủ công cần bổ sung

Bảng 3.7: Kịch bản kiểm thử thủ công trước khi nộp

Mã	Tiền điều kiện	Thao tác	Kết quả mong đợi
MT01	Guest, có truyện approved	Tìm theo từ khóa, mở chi tiết và đọc chương	Không yêu cầu login cho nội dung public; URL đúng slug/chương.
MT02	User đã đăng nhập	Follow, tải lại trang, xem danh sách theo dõi	Trạng thái nhất quán với DB; lỗi mạng có rollback.
MT03	User đọc nhiều chương	Đọc, cuộn, thoát và mở lịch sử	Trở về đúng truyện/chương; không ghi nhầm user.
MT04	Uploader có truyện pending	Thử thêm chương hoặc tự publish	Bị chặn với thông báo đúng; không đổi DB.
MT05	Moderator có truyện pending	Yêu cầu sửa không nhập lý do, sau đó nhập lý do	Lần đầu bị validate; lần sau cập nhật trạng thái, notify và audit.
MT06	User tạo report	Gửi lặp trong khoảng rate limit	Report đầu hợp lệ; request lặp bị hạn chế.
MT07	Admin đăng nhập	Đổi role/status một user	UI cập nhật; audit không chứa password/token.
MT08	Token hết hạn	Mở trang account/admin	API trả 401; frontend xóa phiên và về login không lặp vô hạn.
MT09	Màn hình 375px và desktop	Duyệt Home, Reader, Admin	Không tràn ngang; nút có thể thao tác; chữ đủ đọc.
MT10	Hạ tầng Redis/worker thật	Tạo chương mới đủ điều kiện	Job được xử lý một lần; notification tạo đúng người nhận.

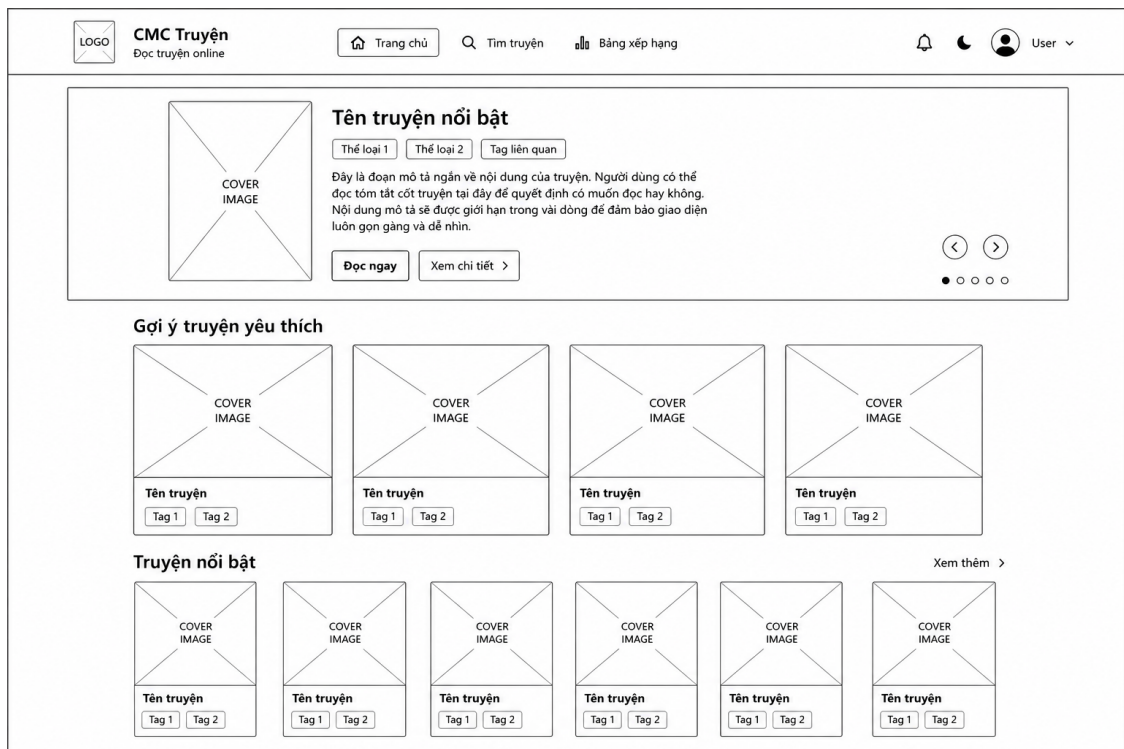
3.7. Giao diện người dùng

Toàn bộ ảnh giao diện trong bản in cần chuyển sang trắng đen hoặc chụp ở chế độ không màu theo yêu cầu, nhưng vẫn phải giữ đủ tương phản để đọc chữ và nhận biết trạng thái. Các khung dưới đây là vị trí chèn ảnh thật; caption

đã được đánh số theo chương và tự động xuất hiện trong danh mục hình.

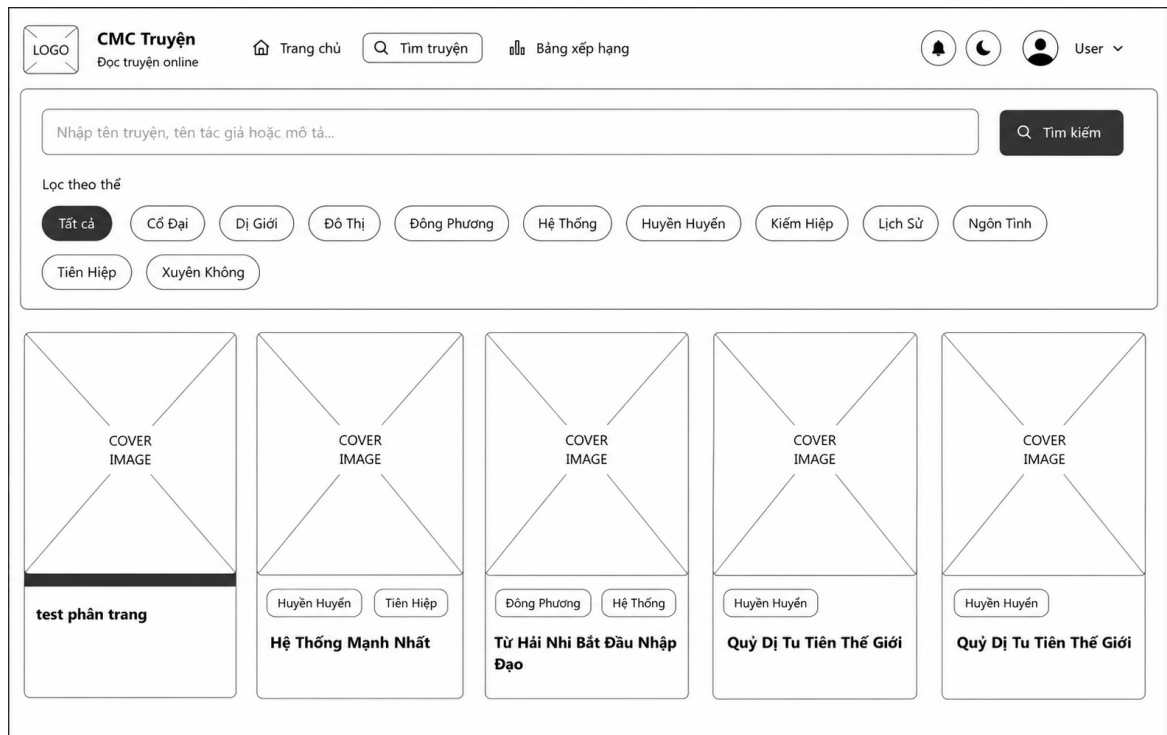
3.7.1. Giao diện khách hàng

Trang chủ cần thể hiện cấu trúc khám phá: điều hướng, nội dung nổi bật và các danh sách chính. Ảnh phải được chụp từ phiên bản chạy thật, không dùng mockup nếu phần mô tả gọi đó là sản phẩm đã triển khai.



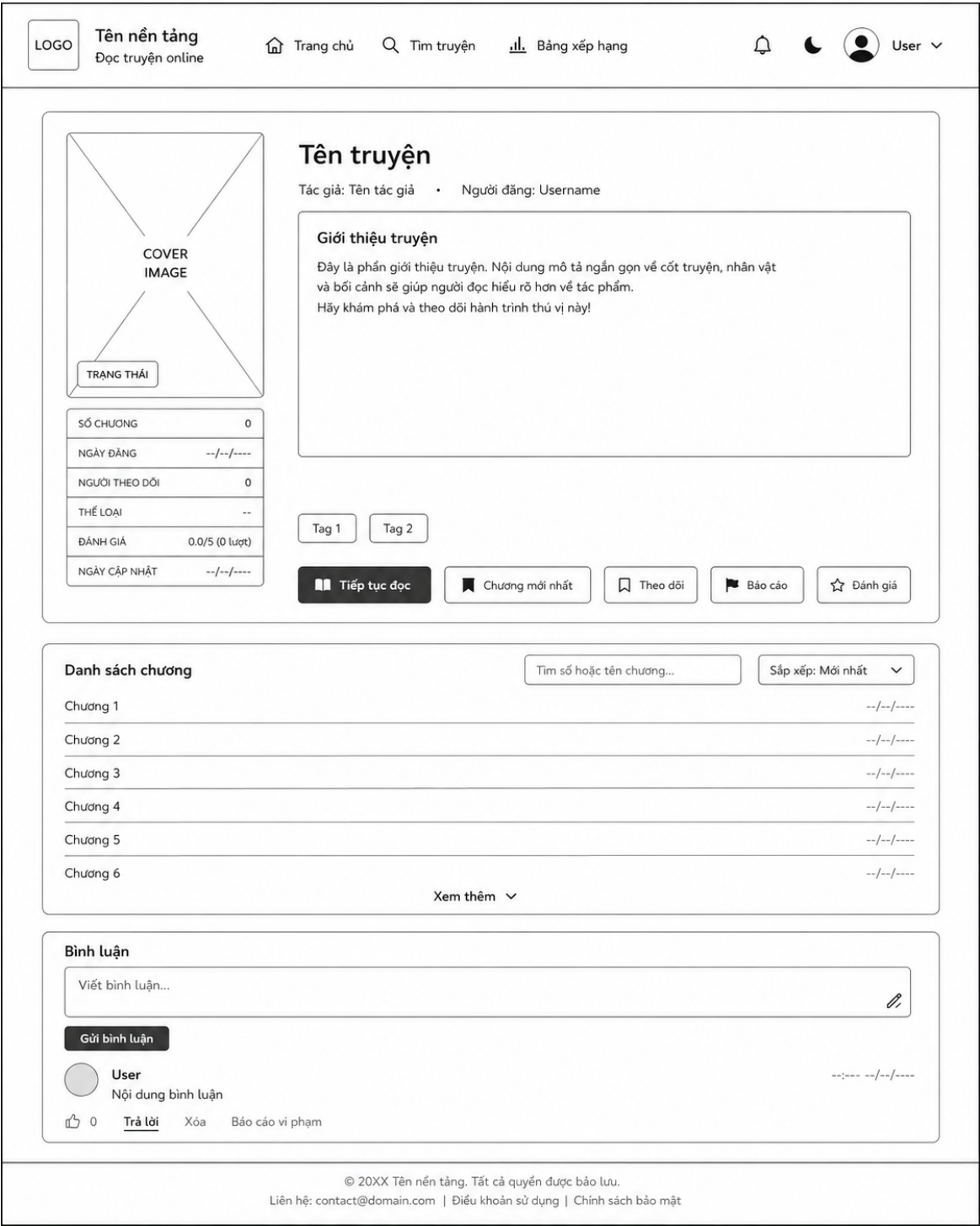
Hình 3.12: Giao diện trang chủ dành cho khách hàng

Trang tìm kiếm/khám phá cần chứng minh bộ lọc, kết quả và trạng thái không có dữ liệu. Nếu chỉ có một ảnh, ưu tiên trạng thái có kết quả và mô tả thêm cách xử lý trạng thái rỗng trong phần thuyết minh.



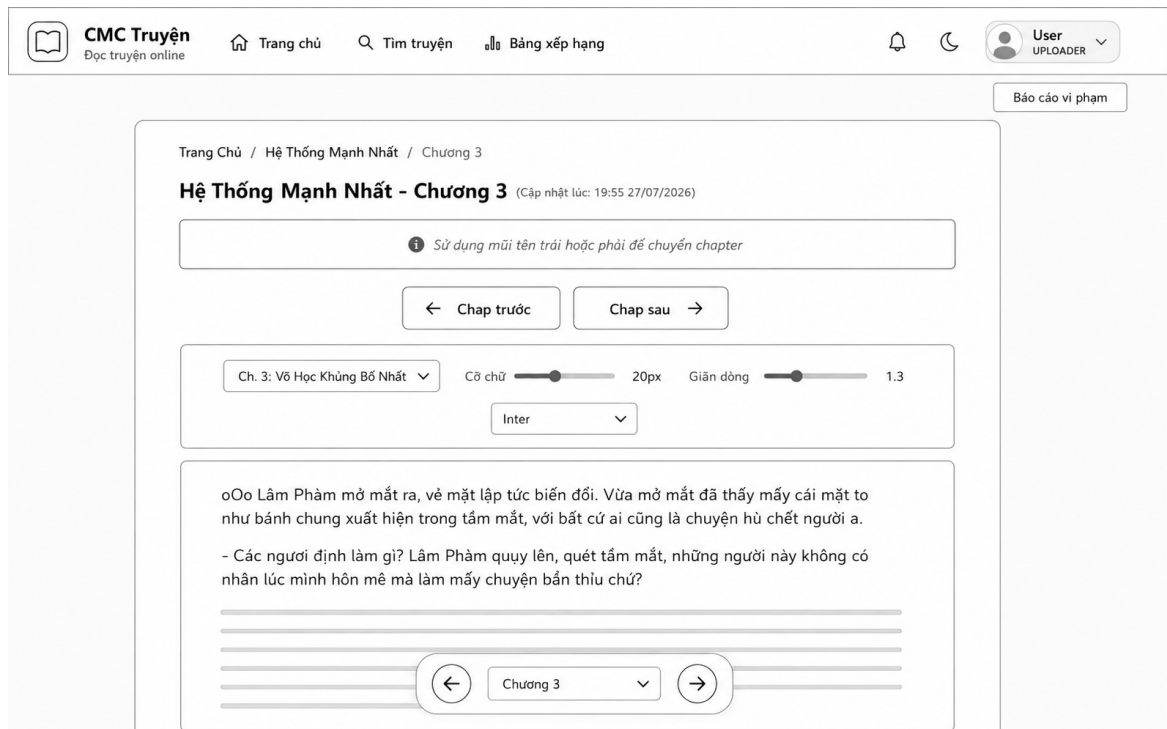
Hình 3.13: Giao diện tìm kiếm và khám phá truyện

Trang chi tiết là điểm nối giữa khám phá và đọc. Ảnh nên có bìa, metadata, mô tả, danh sách chương và hành động follow/rating nếu phiên đăng nhập cho phép.



Hình 3.14: Giao diện chi tiết truyện

Trang đọc chương cần ưu tiên khả năng đọc, điều hướng chương và tùy chọn hiển thị. Vì ảnh trắng đen, phải kiểm tra đường biên/nút không phụ thuộc duy nhất vào màu.



Hình 3.15: Giao diện đọc chương và tùy chọn đọc

Trang tài khoản thể hiện giá trị sau đăng nhập như truyện theo dõi, lịch sử hoặc cài đặt. Ảnh không được lộ email, token hay dữ liệu cá nhân thật nếu chưa được đồng ý.

LOGO

Tên nền tảng
Đọc truyện online

Trang chủ

Tim truyện

Bảng xếp hạng

🌙

User

Tiêu đề trang

Mô tả ngắn

Tab 1

Tab 2

Tab 3

Tên người dùng

Email

Mô tả ngắn

Nút hành động

Chỉnh sửa hồ sơ

Thông tin này sẽ hiển thị với những người dùng khác.

Tên hiển thị

Giới thiệu bản thân

Ảnh đại diện

Chọn tệp

Chưa chọn tệp

Nút hành động

Cài đặt đọc

Tùy chọn 1

--px

Tùy chọn 2

Tùy chọn 3

Tùy chọn 4

☒

Đổi mật khẩu

Sử dụng mật khẩu mạnh và không dùng lại ở nơi khác.

Mật khẩu hiện tại

Mật khẩu mới

Xác nhận mật khẩu

Nút hành động

© 20XX Tên nền tảng. Tất cả quyền được bảo lưu.

Liên hệ

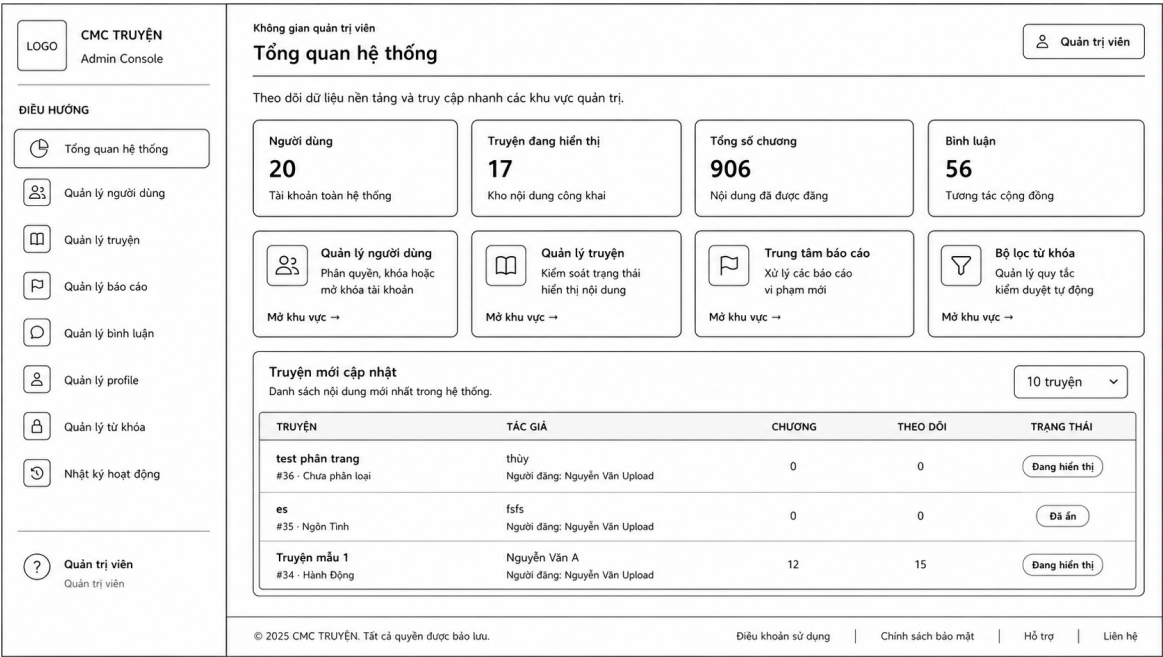
Điều khoản sử dụng

Chính sách bảo mật

Hình 3.16: Giao diện cài đặt cá nhân

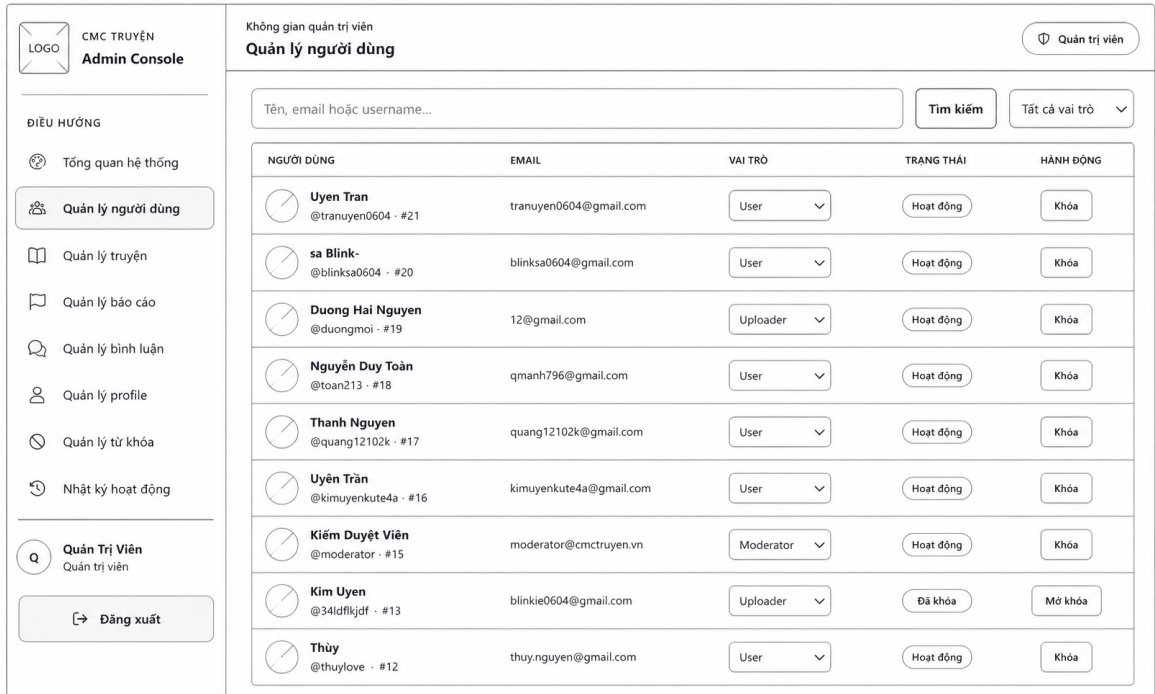
3.7.2. Giao diện Admin và Moderator

Khu vực quản trị cần thể hiện rõ layout riêng, sidebar, tiêu đề trang và trạng thái active. Dashboard nên dùng số liệu demo nhất quán với database của lần chụp; không tự điền số đẹp vào ảnh.



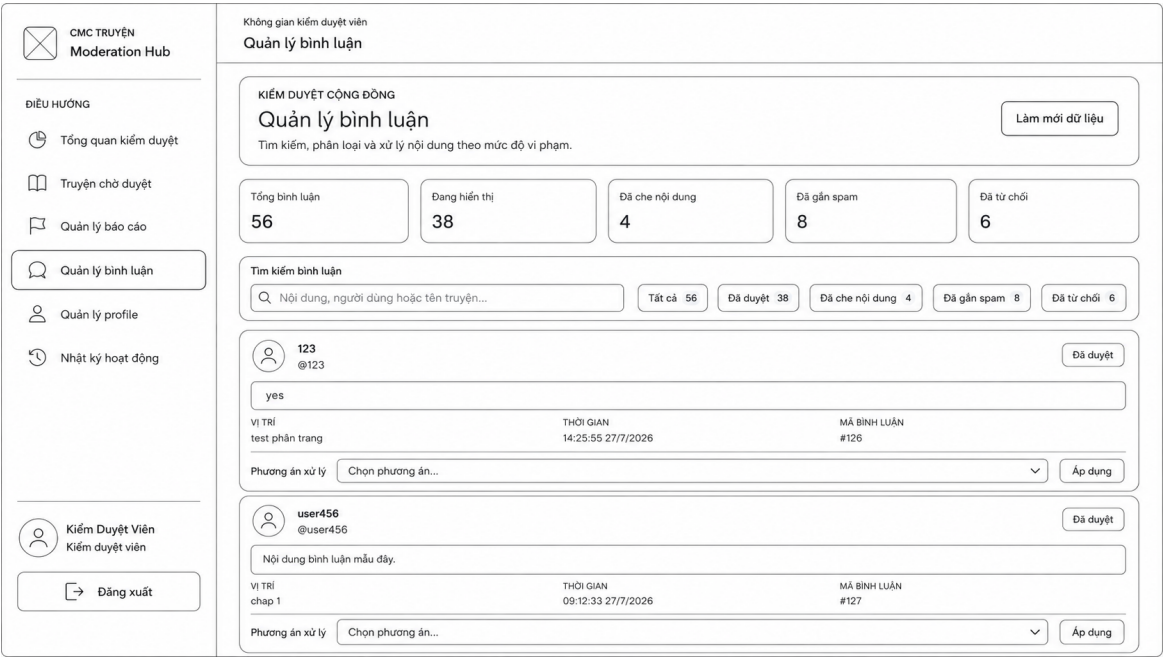
Hình 3.17: Giao diện dashboard quản trị

Màn hình user management cần chứng minh tìm kiếm, role, trạng thái và thao tác quản trị. Nên tránh chụp danh sách tài khoản thật; dữ liệu seed/demo là phù hợp hơn.



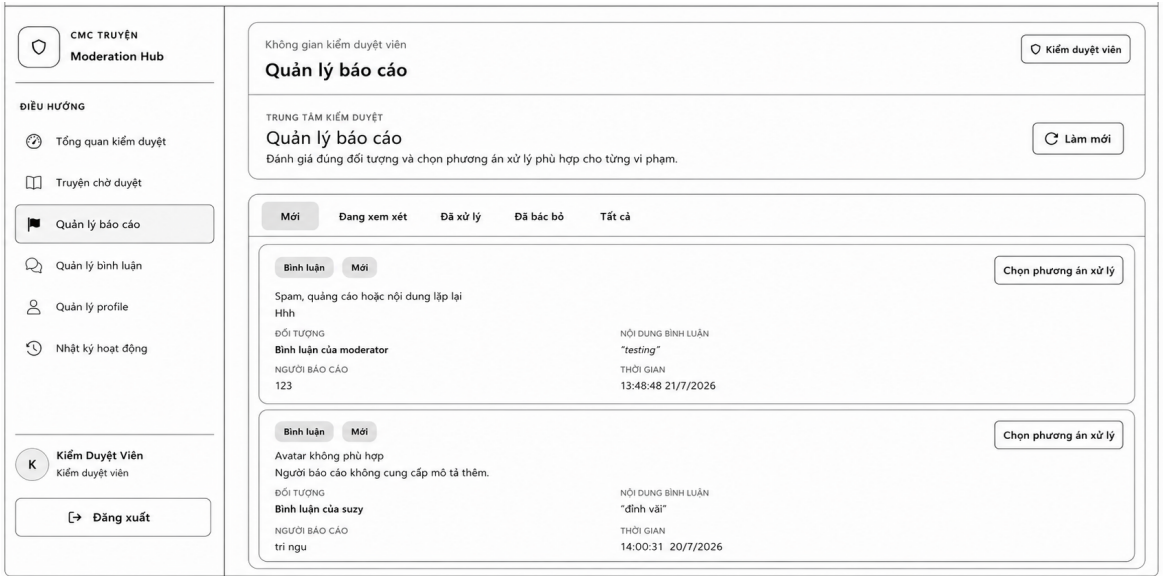
Hình 3.18: Giao diện quản lý người dùng

Hàng chờ truyện là màn hình quan trọng để chứng minh workflow không chỉ tồn tại ở backend. Ảnh cần có trạng thái pending và các hành động duyệt/yêu cầu sửa/từ chối.



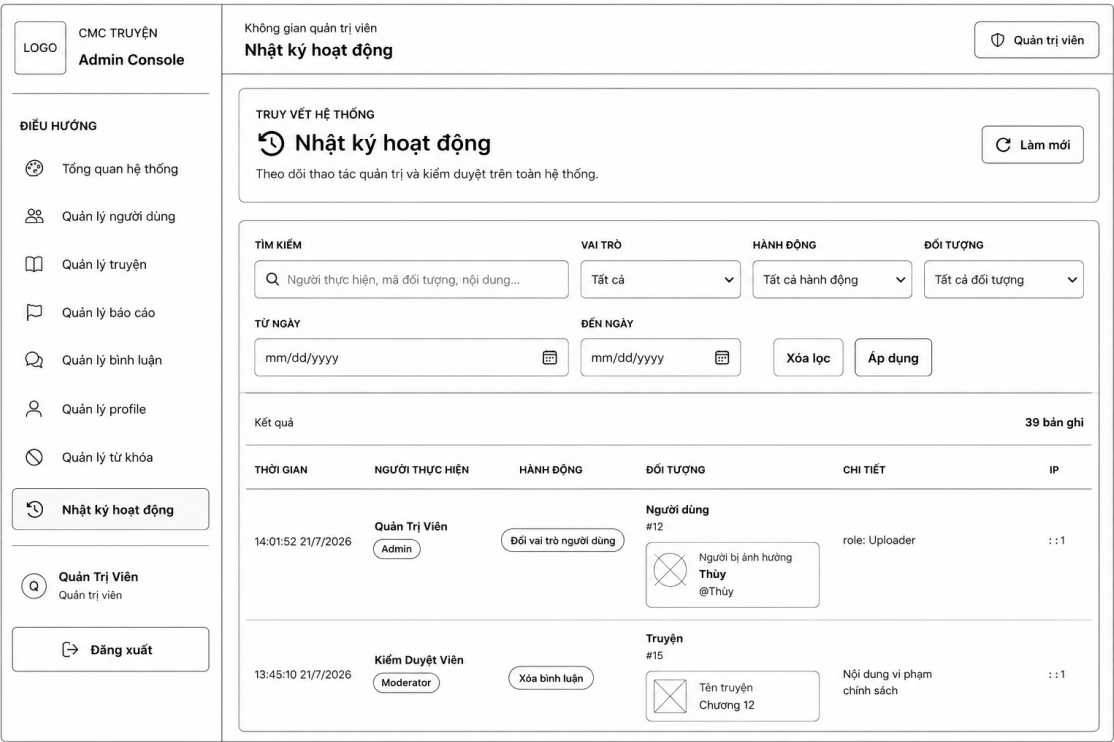
Hình 3.19: Giao diện quản lý bình luận

Màn hình report thể hiện khả năng lọc và xử lý vi phạm. Nên chụp trạng thái trước khi xử lý; nếu cần ảnh sau xử lý, tạo report demo riêng để không làm sai dữ liệu của nhóm.



Hình 3.20: Giao diện quản lý báo cáo vi phạm

Audit log chứng minh tính truy vết. Ảnh phải cho thấy actor, action, entity và thời điểm nhưng không chứa token, password hay request body nhạy cảm.



Hình 3.21: Giao diện nhật ký thao tác quản trị

3.8. Đánh giá, hạn chế và hướng hoàn thiện

3.8.1. Điểm đạt được

Source đã hình thành một sản phẩm full-stack có phạm vi tương đối rộng: route SPA cho nhiều vai trò, API phân lớp, PostgreSQL với ràng buộc/index, workflow kiểm duyệt, report, audit, queue/worker và kiểm thử tự động. Kết quả 72 test pass và build production thành công là bằng chứng kỹ thuật cụ thể. Tài liệu kiến trúc/API trong repository cũng giúp truy vết quyết định.

3.8.2. Hạn chế cần công khai

- migration/schema phân tán, thiếu DDL bad_words trong phần source rà soát và còn DDL chạy lúc startup;
- test log có Redis ECONNREFUSED và Jest force-exit, cho thấy tear-down/tích hợp queue chưa sạch;
- chưa có kết quả E2E, kiểm thử tải, security audit và accessibility audit trong lần xác minh này;
- bundle frontend vượt ngưỡng cảnh báo 500 kB;
- link deploy, link commit, video và ảnh giao diện chưa được cung cấp nên báo cáo để trống.

3.8.3. Thứ tự ưu tiên hoàn thiện

Nhóm cần hoàn tất metadata, phân công, URL và ảnh thật; sau đó chuẩn hóa migration, xử lý Redis/open handles, chạy E2E và tối ưu bundle.

3.9. Kết luận chương

Chương 3 đã trình bày kiến trúc SPA + REST, cách tổ chức frontend, backend, cơ sở dữ liệu, triển khai và kiểm thử. Các hạn chế còn lại được tổng hợp thành danh sách ưu tiên hoàn thiện.

KẾT LUẬN

Báo cáo đã trình bày bài toán, phạm vi, luồng hoạt động theo vai trò và quá trình triển khai CMC Truyện bằng React, Express, PostgreSQL và Redis/BullMQ. Sản phẩm đã hình thành các nhóm chức năng chính: khám phá và đọc truyện, quản lý tài khoản, tương tác, đăng nội dung, kiểm duyệt và quản trị.

Tại lần xác minh được ghi nhận trong dự án, 72/72 kiểm thử đã đạt và frontend build production thành công. Kết quả này cho thấy các luồng cốt lõi đã được kiểm tra ở mức mã nguồn. Tuy nhiên, hệ thống vẫn cần đồng bộ schema/migration với ERD, xử lý kết nối Redis trong môi trường test, tối ưu bundle và bổ sung kiểm thử E2E trên hạ tầng thật.

Trước khi nộp, nhóm cần điền các liên kết còn trống, bổ sung ảnh giao diện đúng phiên bản triển khai và cập nhật kết quả kiểm thử thủ công. Trong giai đoạn tiếp theo, nhóm định hướng chuẩn hóa migration, hoàn thiện E2E, tối ưu hiệu năng frontend và bổ sung giám sát khi vận hành.

TÀI LIỆU THAM KHẢO

- [1] Nhóm phát triển CMC Truyền, *Đặc tả yêu cầu và phạm vi chức năng dự án CMC Truyền*, tài liệu nội bộ trong thư mục docs, phiên bản dùng để nộp, 2026.
- [2] Nhóm phát triển CMC Truyền, *Tài liệu kiến trúc, API và cấu hình triển khai dự án CMC Truyền*, tài liệu nội bộ trong kho mã nguồn, phiên bản dùng để nộp, 2026.
- [3] Meta Open Source, *React Documentation – Learn React*, <https://react.dev/learn>, truy cập ngày 26/07/2026.
- [4] VoidZero Inc. và cộng đồng Vite, *Vite Guide – Getting Started*, <https://vite.dev/guide/>, truy cập ngày 26/07/2026.
- [5] OpenJS Foundation, *Express.js Guide – Routing*, <https://expressjs.com/en/guide/routing/>, truy cập ngày 26/07/2026.
- [6] PostgreSQL Global Development Group, *PostgreSQL Documentation – Constraints*, <https://www.postgresql.org/docs/current/ddl-constraints.html>, truy cập ngày 26/07/2026.
- [7] M. Jones, J. Bradley và N. Sakimura, *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force, 2015, <https://datatracker.ietf.org/doc/html/rfc7519>.
- [8] Redis, *Redis Documentation*, <https://redis.io/docs/latest/>, truy cập ngày 26/07/2026.
- [9] Taskforce.sh, *BullMQ Documentation*, <https://docs.bullmq.io/>, truy cập ngày 26/07/2026.
- [10] OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, <https://owasp.org/www-project-top-ten/>, truy cập ngày 26/07/2026.